



First Steps in Reinforcement Learning with MATLAB

Gareth Thomas
16 April, 2020

4 Things to Remember

- Reinforcement Learning is also possible with MATLAB
- Simulink & Simscape can help model the environment
- Not everything should be solved with Machine Learning
- Follow your passion and keep learning



Motivation

- Many people use Python as a tool of choice
- As a Control Engineers wanted to learn more about Reinforcement Learning (never studied it at University)
- Understand some drawbacks of Reinforcement Learning
- Most people succeed in life when they realize that University is just the beginning



Why Me?

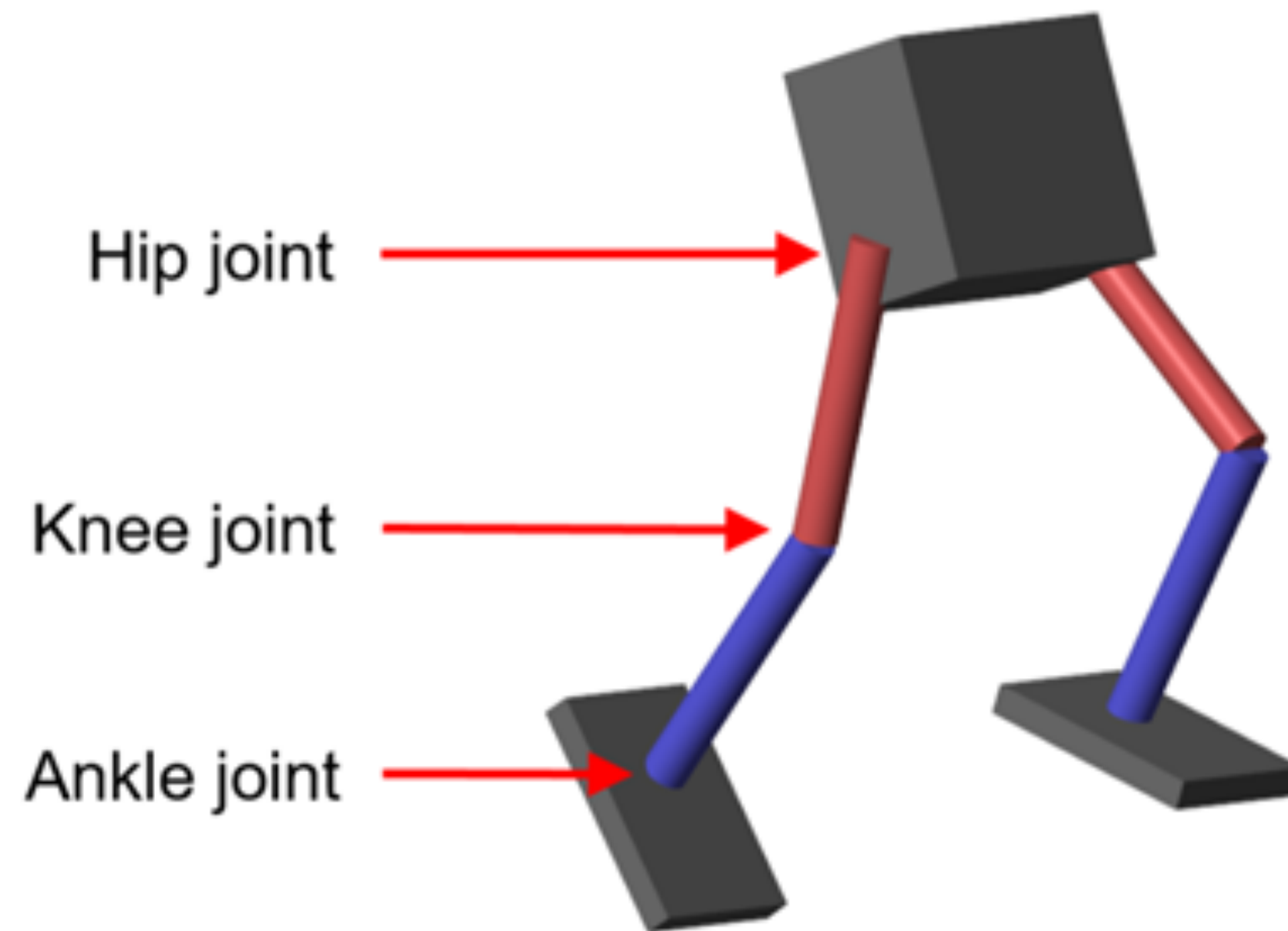
**Control Engineer trying to figure out if
Machine Learning can help!**

(also a co-founder of my own company)

**Just became a Dad
and I noticed how my son started to walk...**



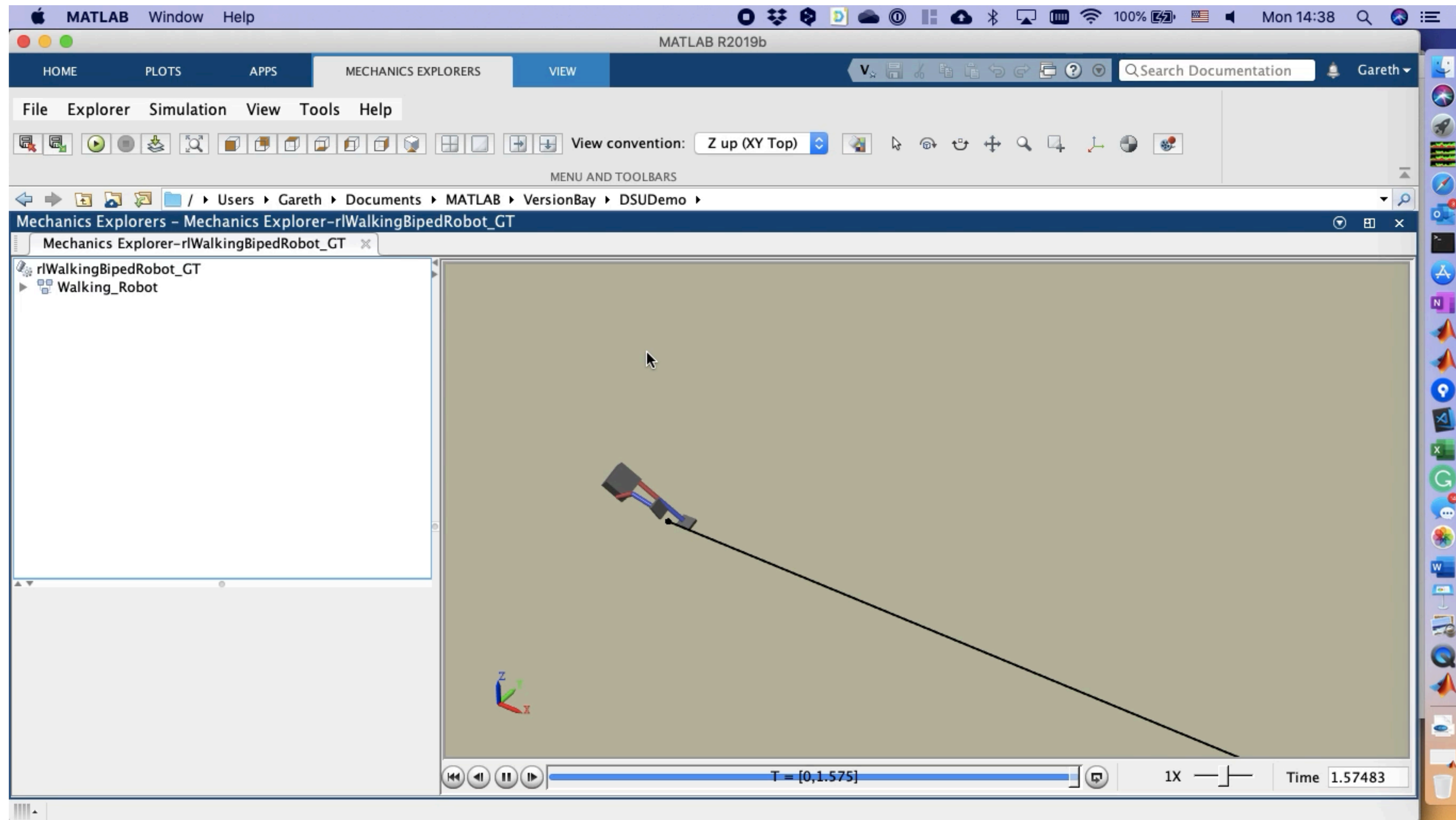
Problem To Be Solved



**Make a robot walk
in a
straight line
using minimal
control effort**



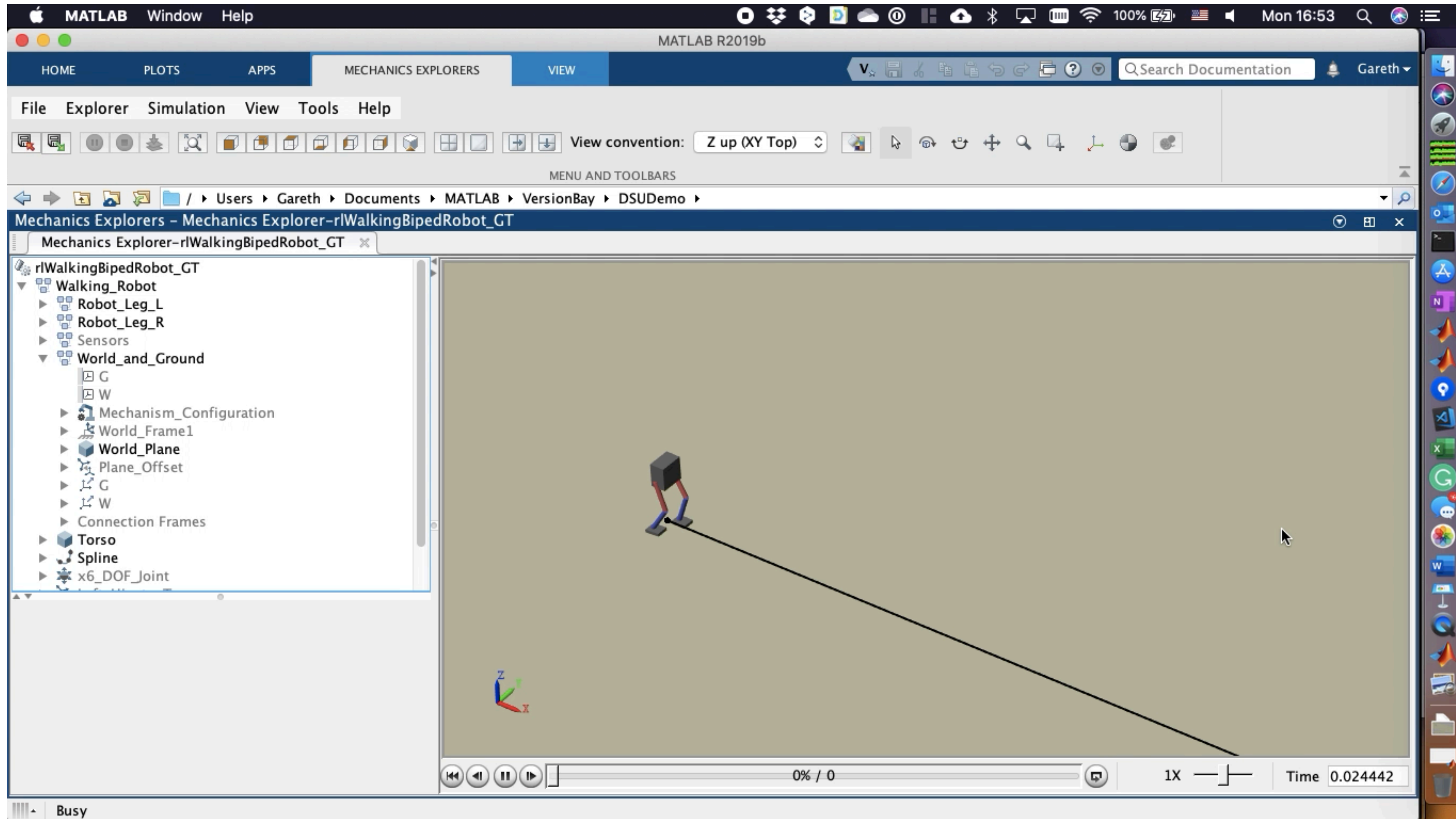
Example In MATLAB



First Try



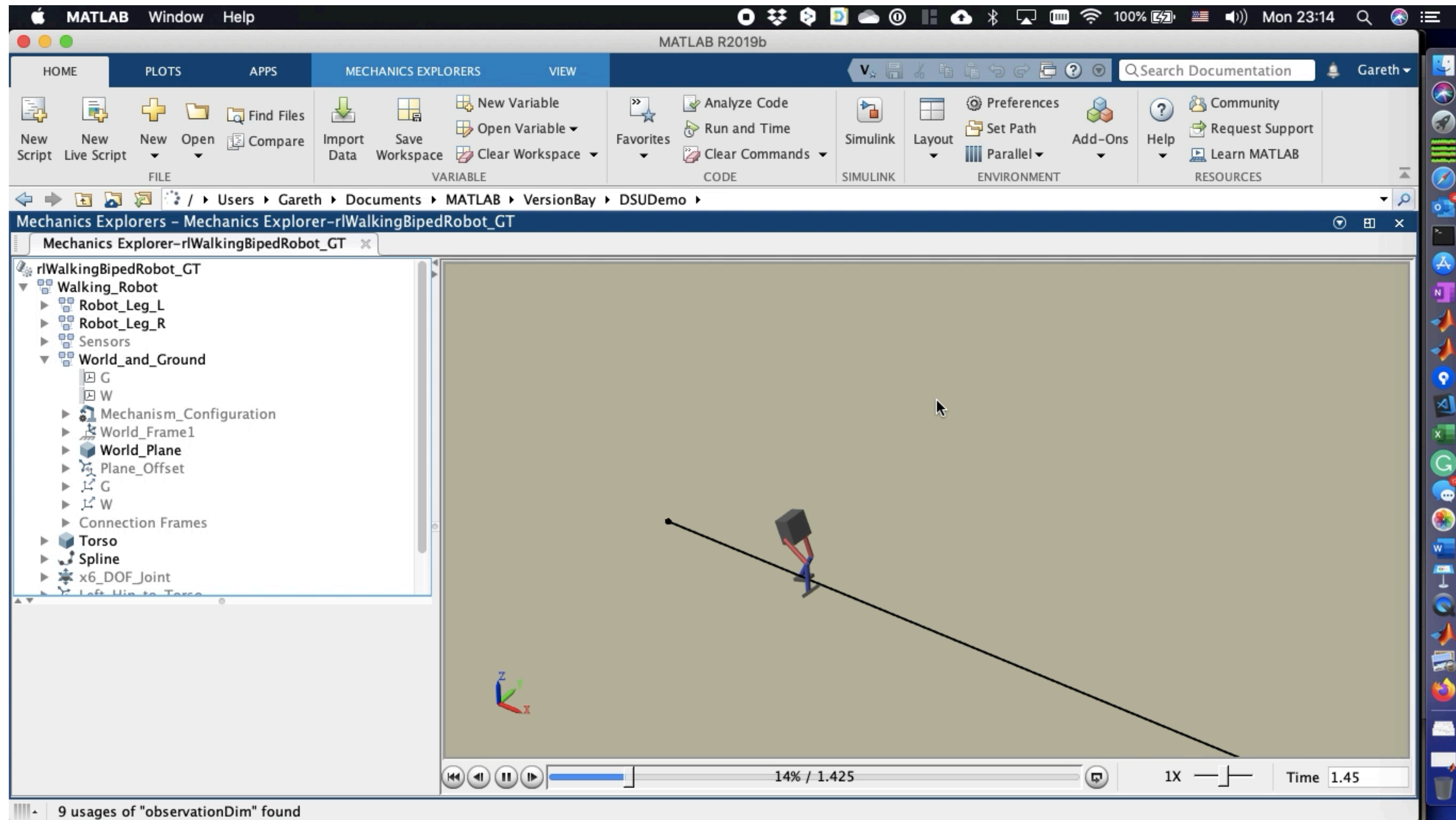
Example In MATLAB



2 hours later



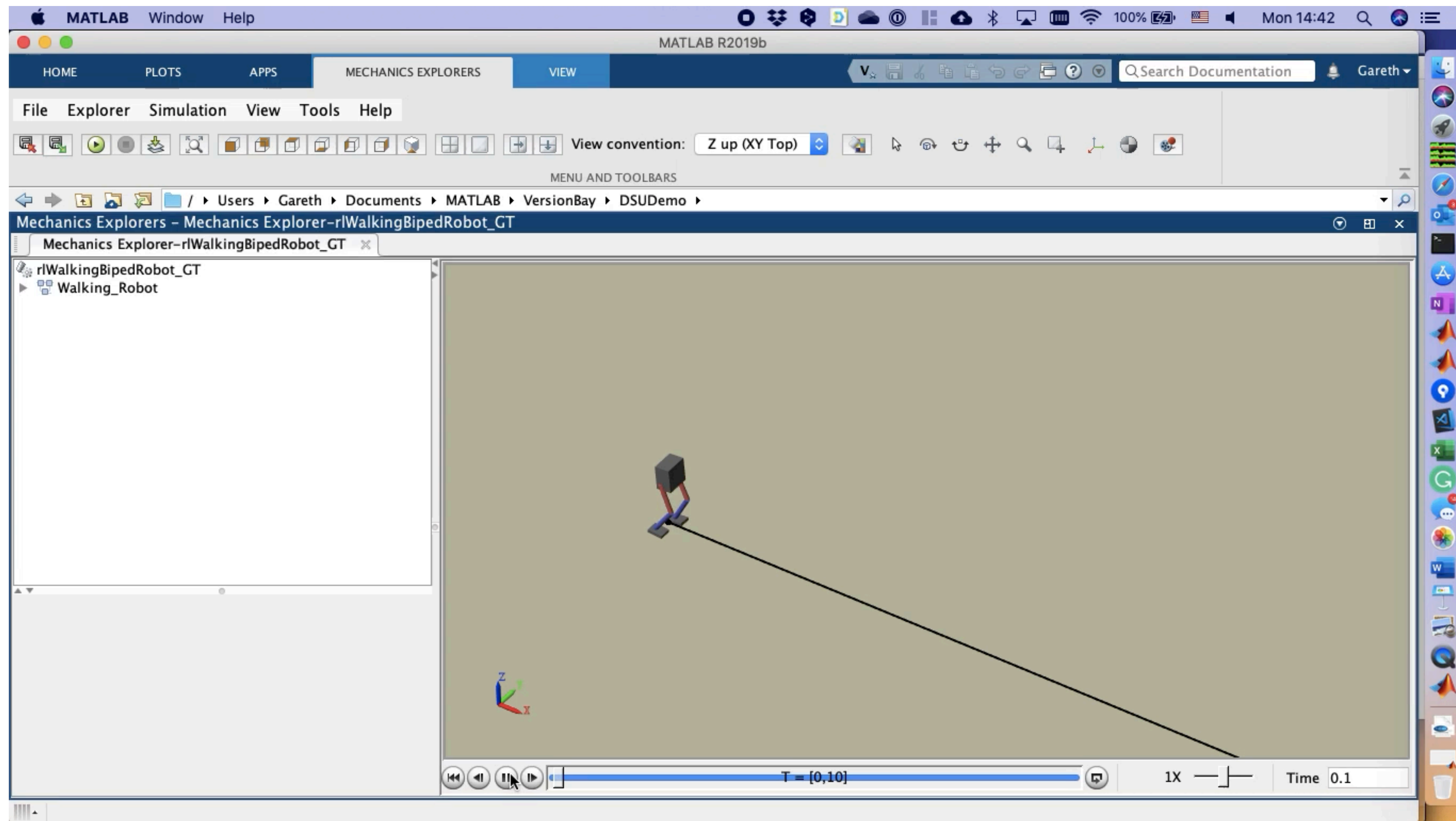
Example In MATLAB



7 hours later



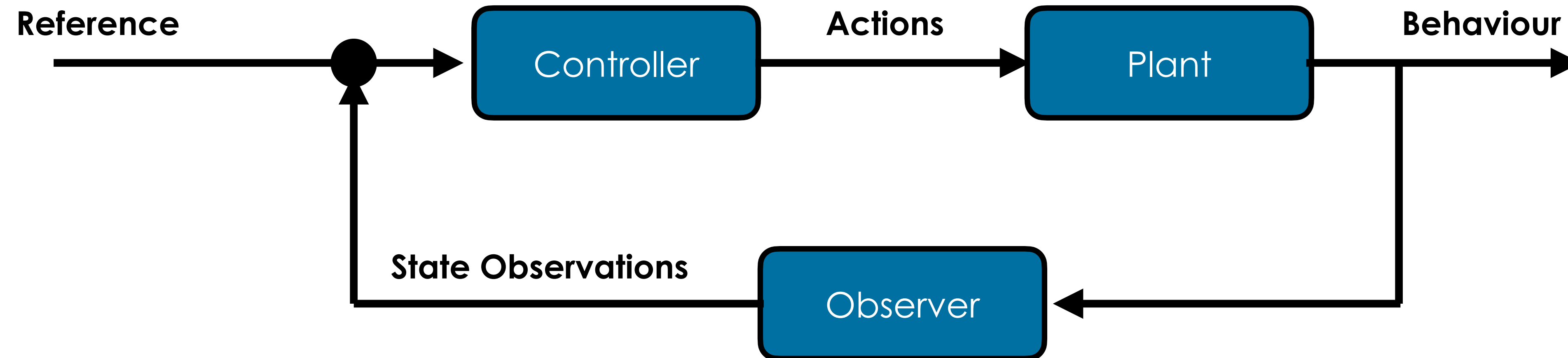
Example In MATLAB



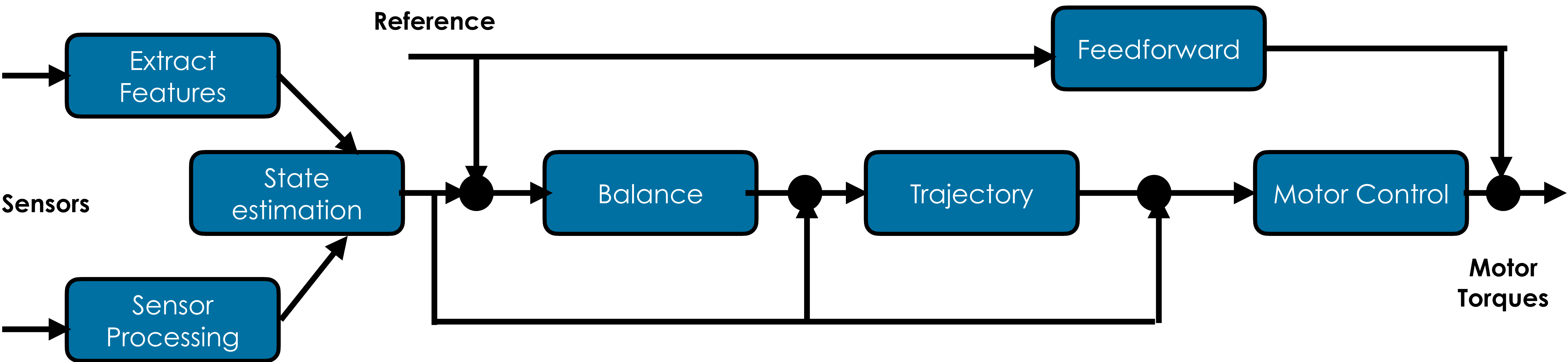
~2 Days Later



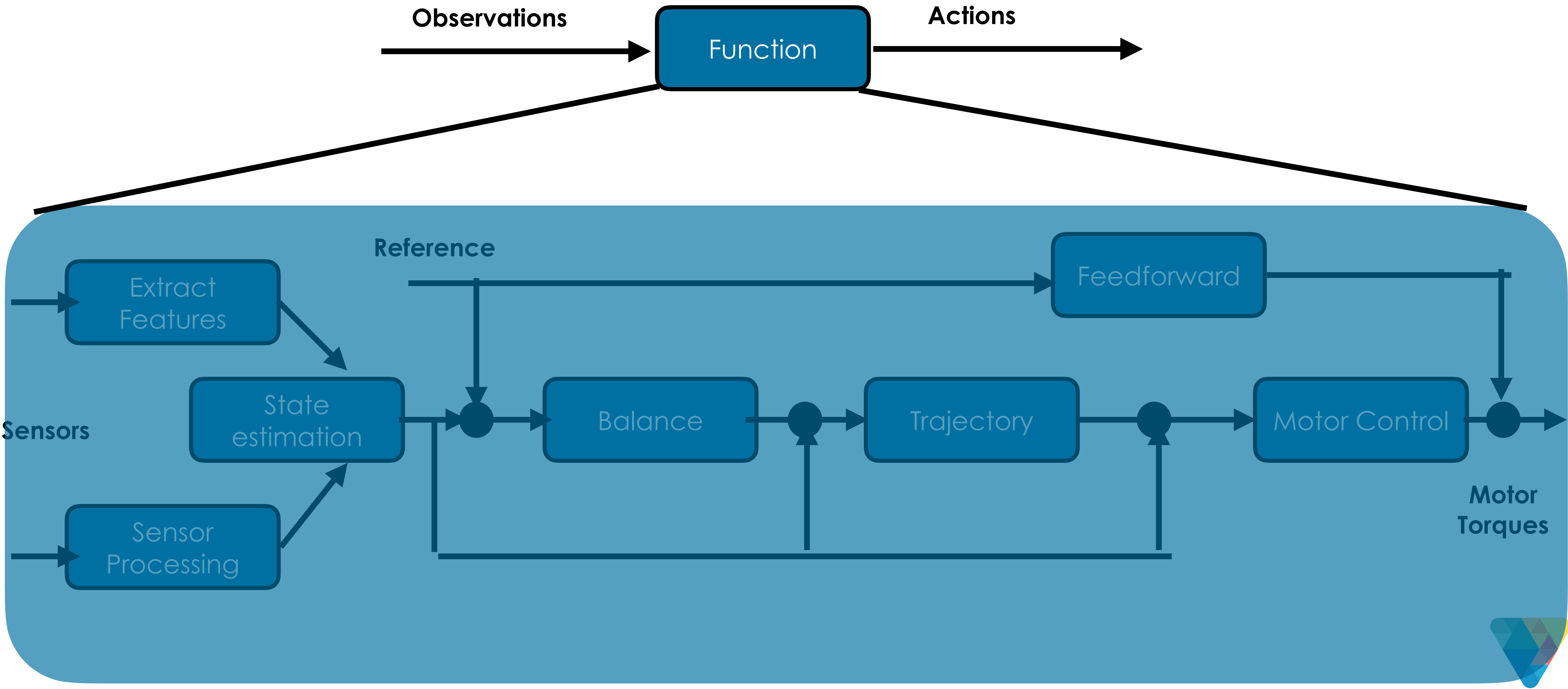
Why Machine Learning?



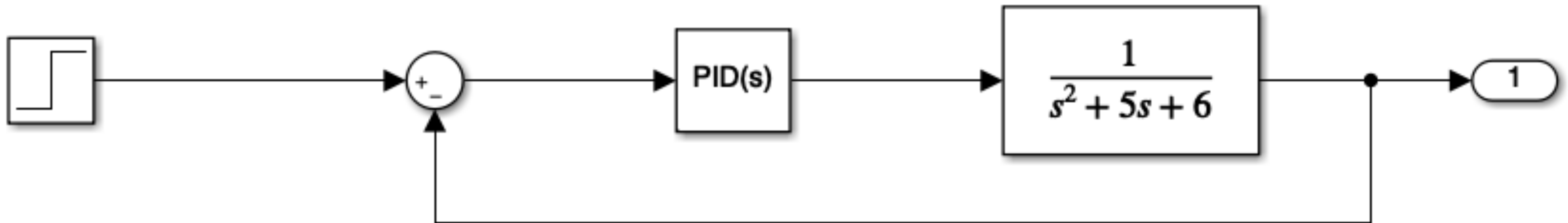
Why Machine Learning?



Why Machine Learning?



Introduction to Simulink



Give a Short Demo



Types of Machine Learning

Unsupervised
Learning

Supervised
Learning

Reinforcement
Learning



Types of Machine Learning

Unsupervised Learning

K-Means

Hierarchical

Gaussian Mixture

Neural Networks

Hidden Markov Model

Supervised Learning

Support Vector Machines

Naive Bayes

Nearest Neighbor

Linear Regression

Decision Trees

Neural Networks

Reinforcement Learning

Types of Machine Learning

Unsupervised Learning

K-Means

Hierarchical

Gaussian Mixture

Neural Networks

Hidden Markov Model

Supervised Learning

Support Vector Machines

Naive Bayes

Nearest Neighbor

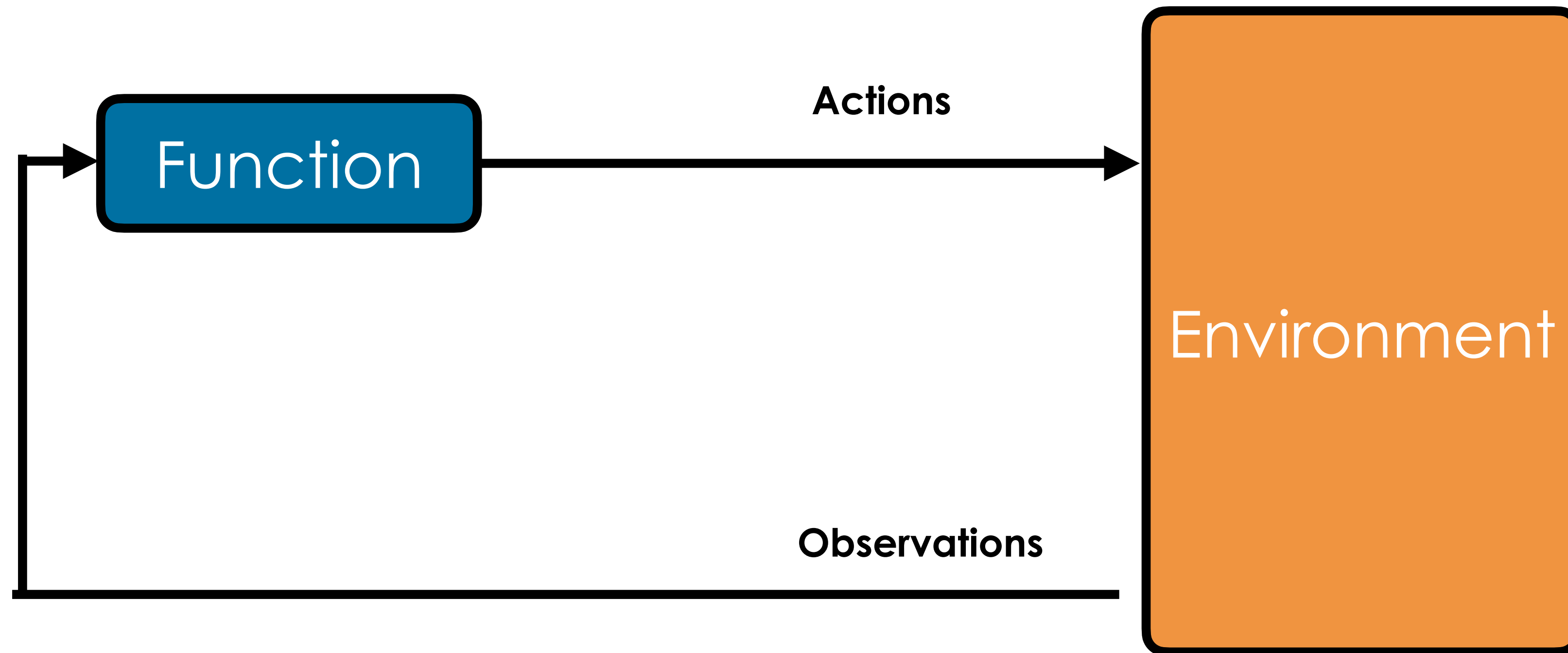
Linear Regression

Decision Trees

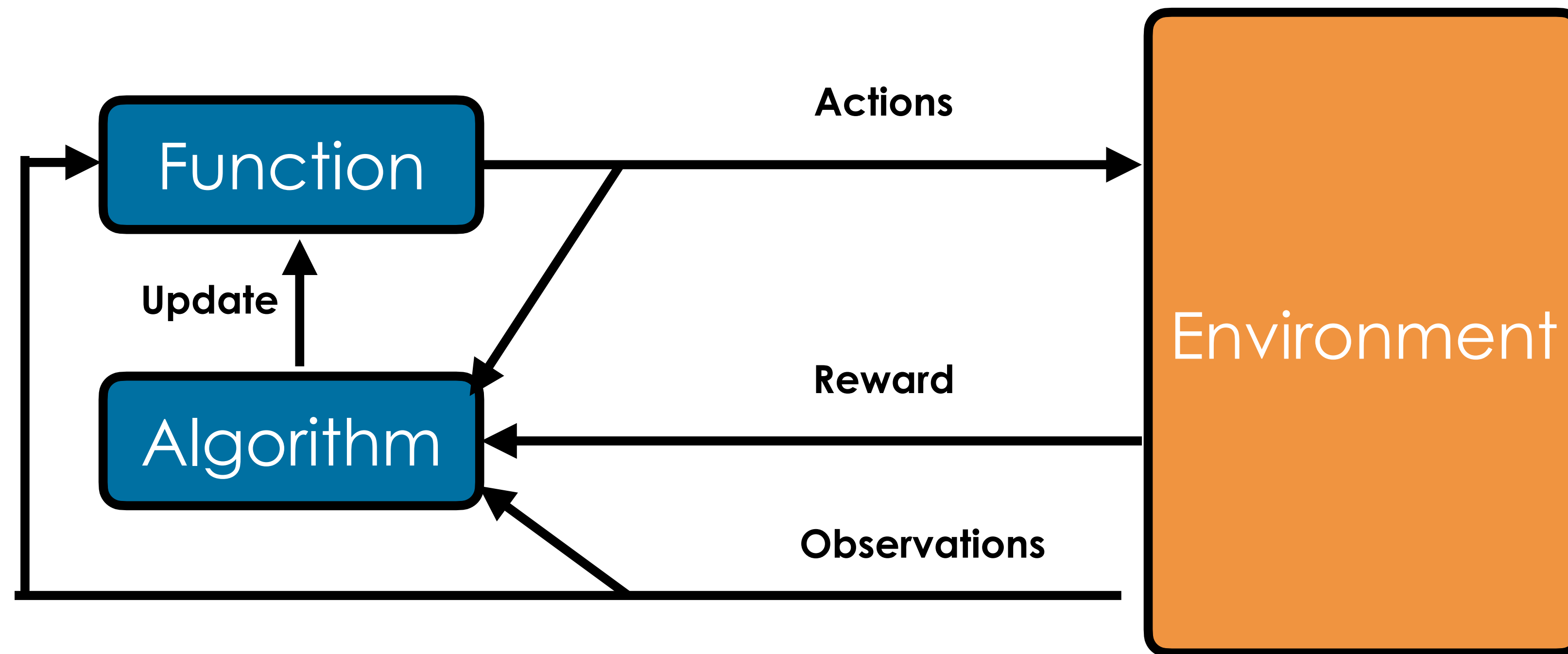
Neural Networks

Reinforcement Learning

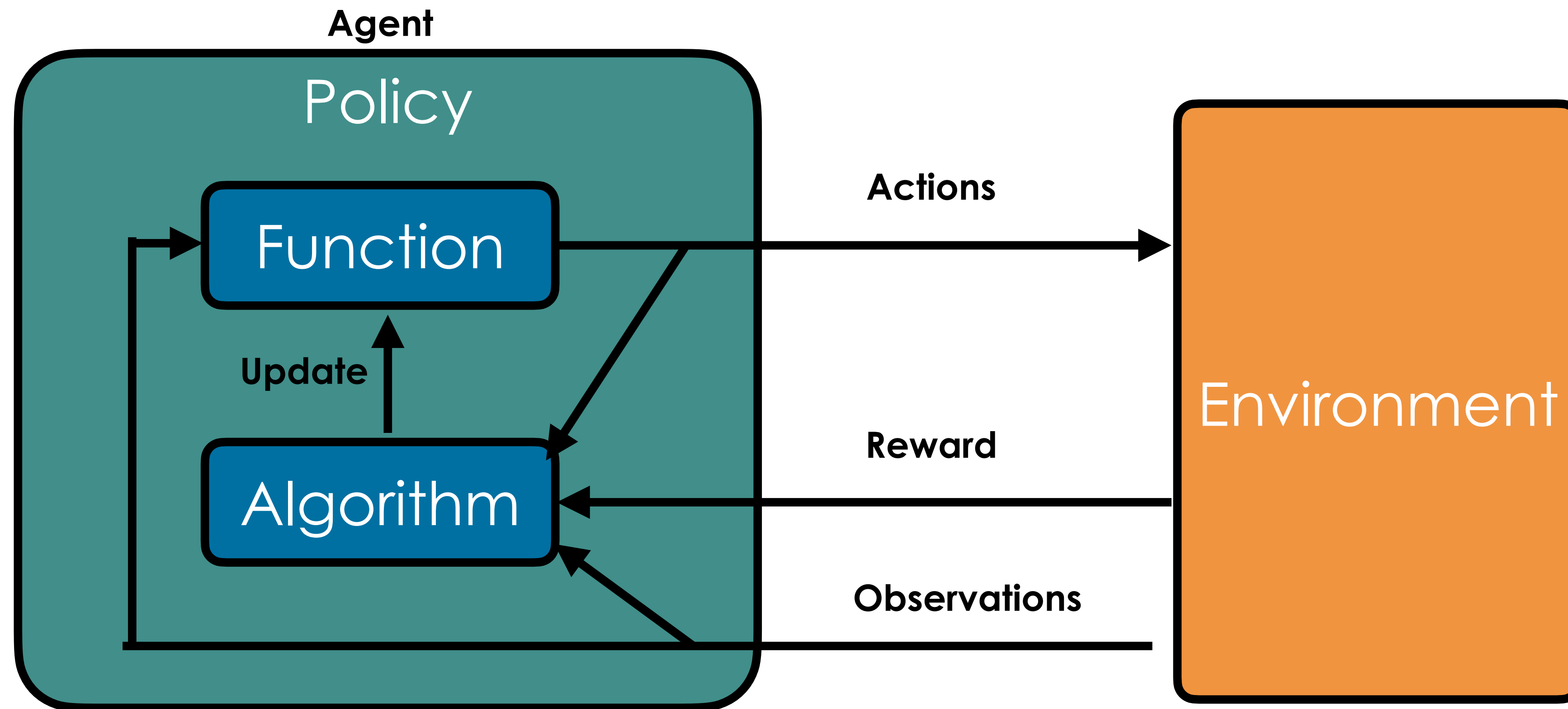
Reinforcement Learning



Reinforcement Learning

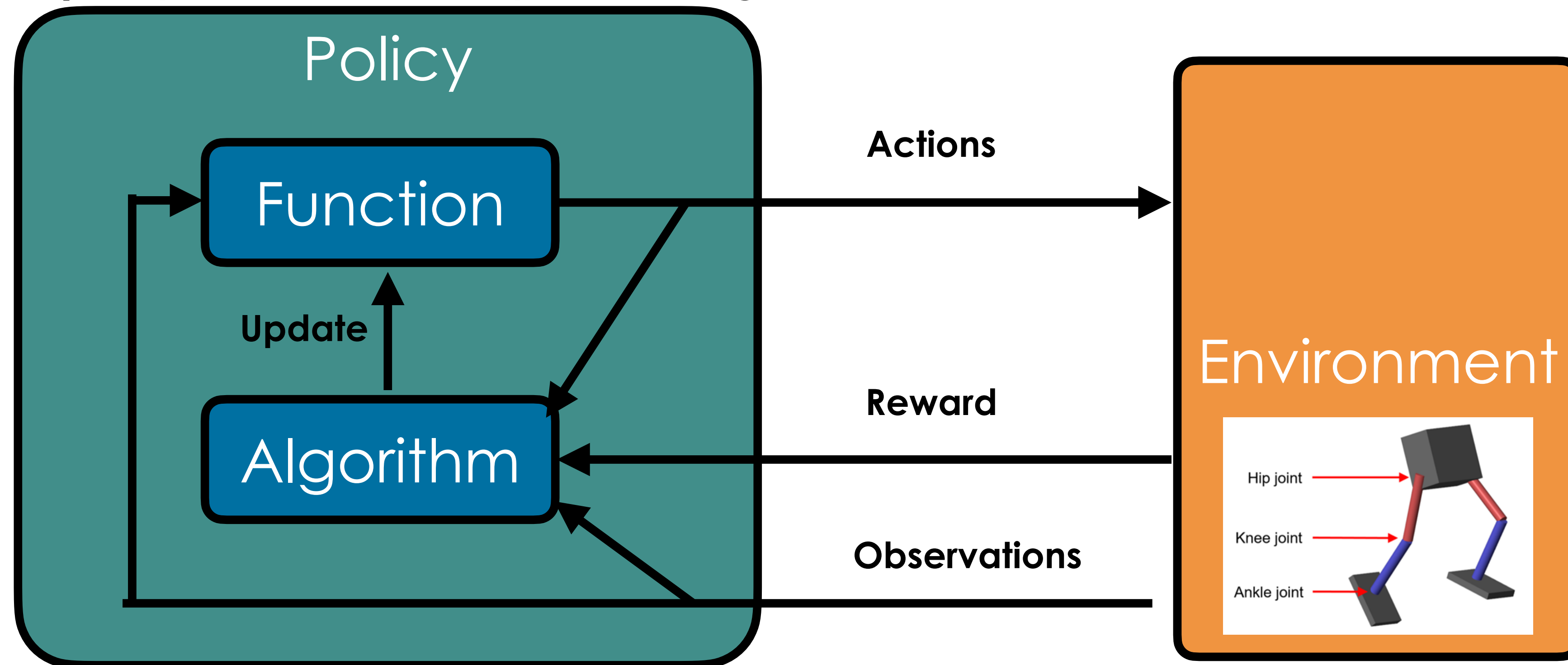


Reinforcement Learning



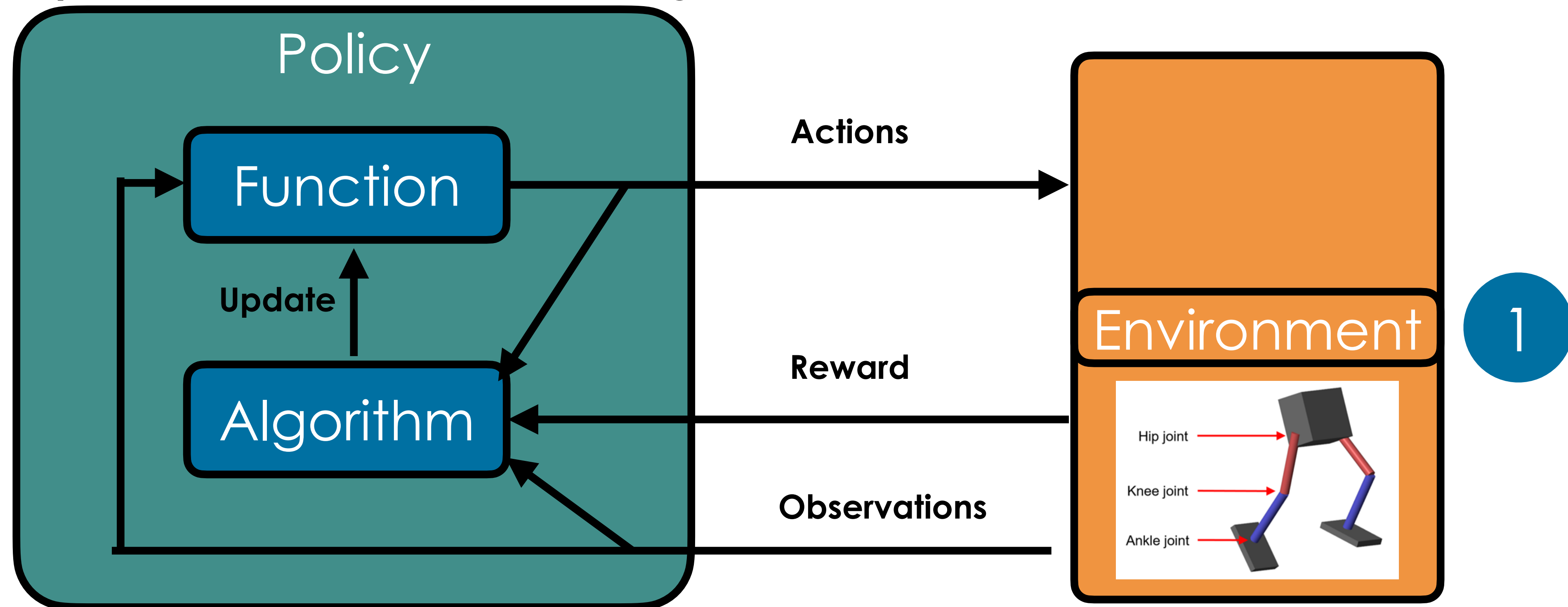
Reinforcement Learning

(DDPG) Deep Deterministic Policy Gradient Agent



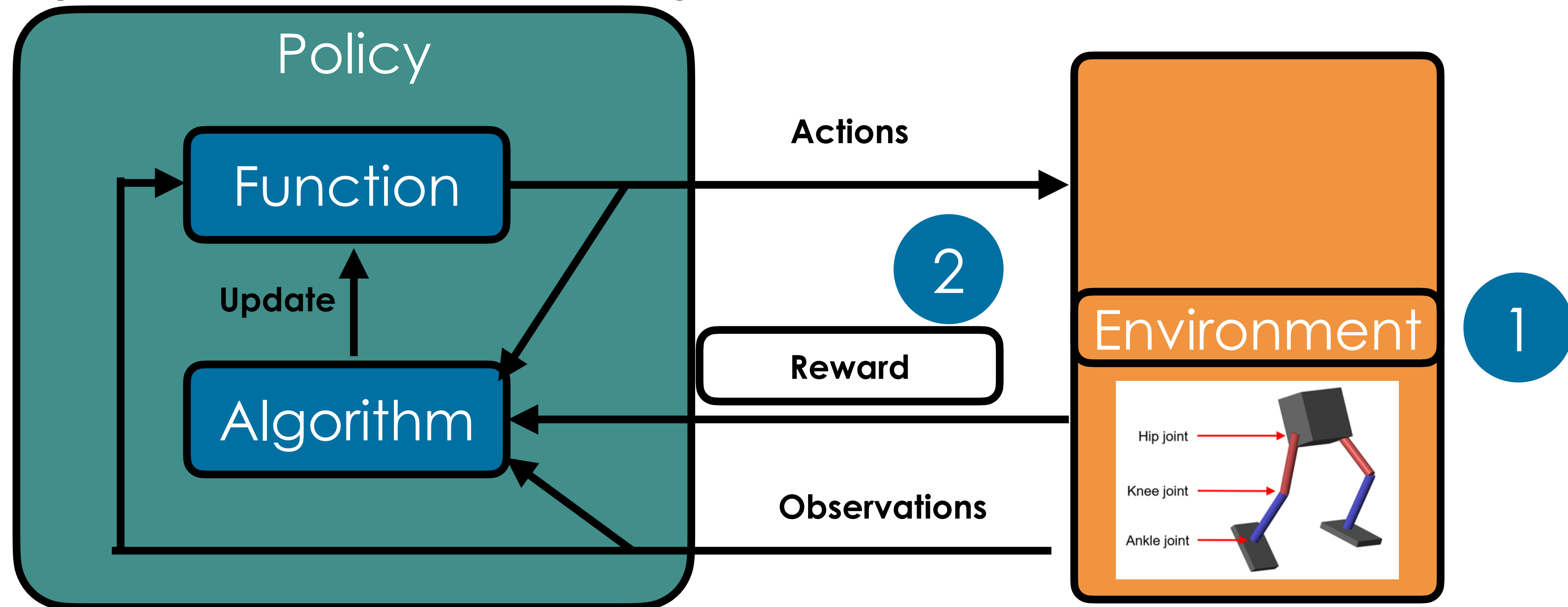
Reinforcement Learning

(DDPG) Deep Deterministic Policy Gradient Agent



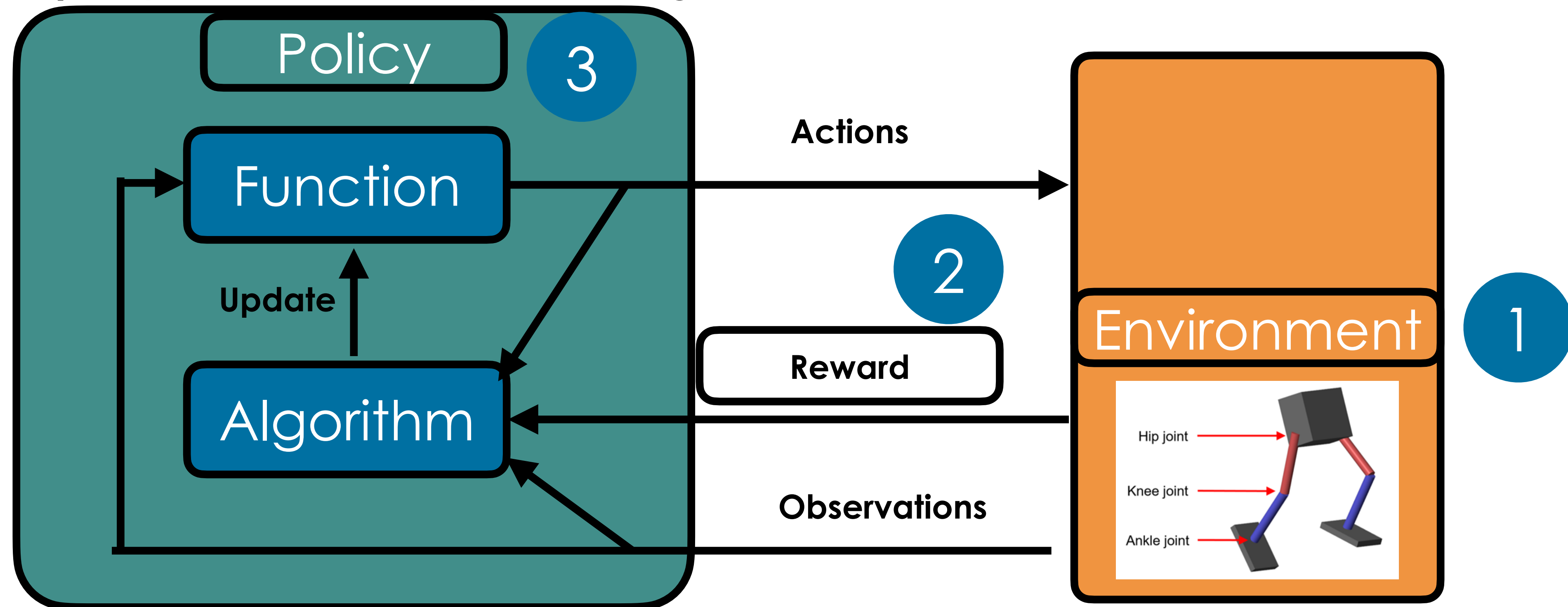
Reinforcement Learning

(DDPG) Deep Deterministic Policy Gradient Agent



Reinforcement Learning

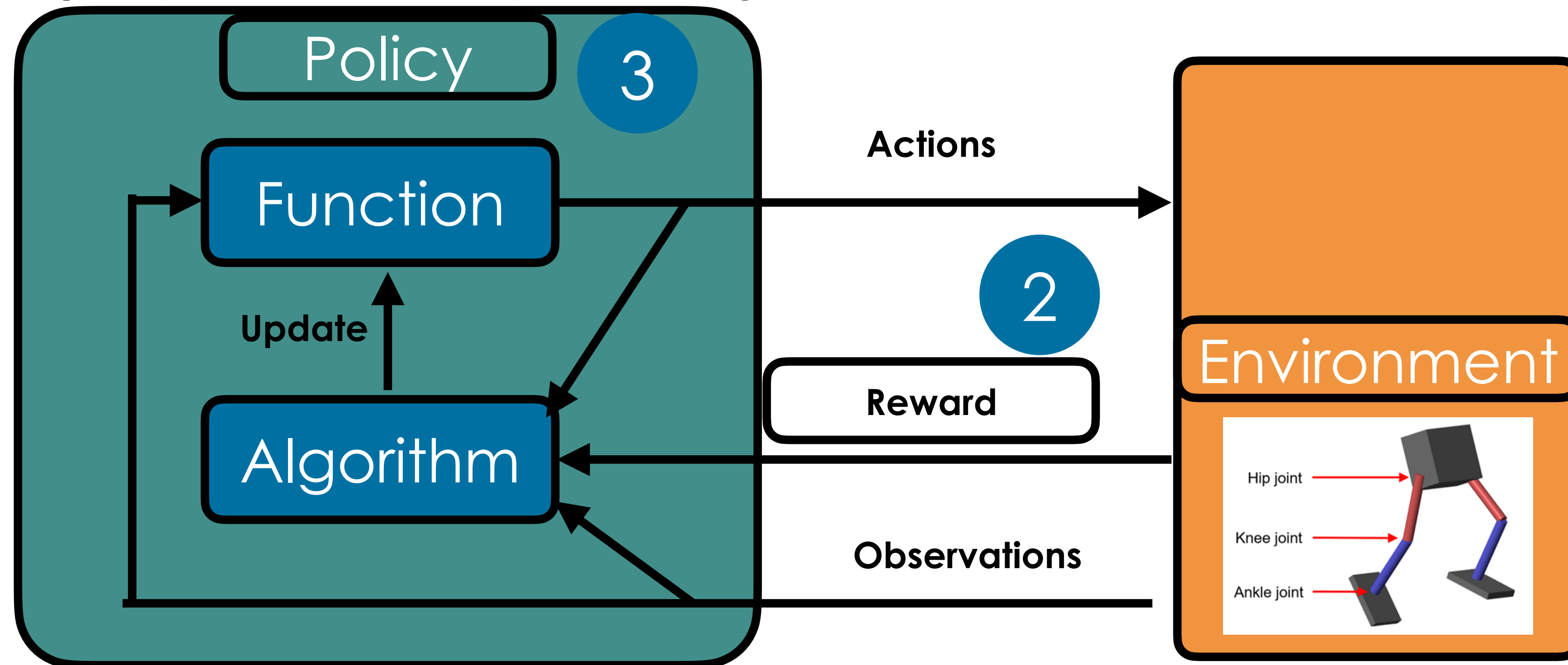
(DDPG) Deep Deterministic Policy Gradient Agent



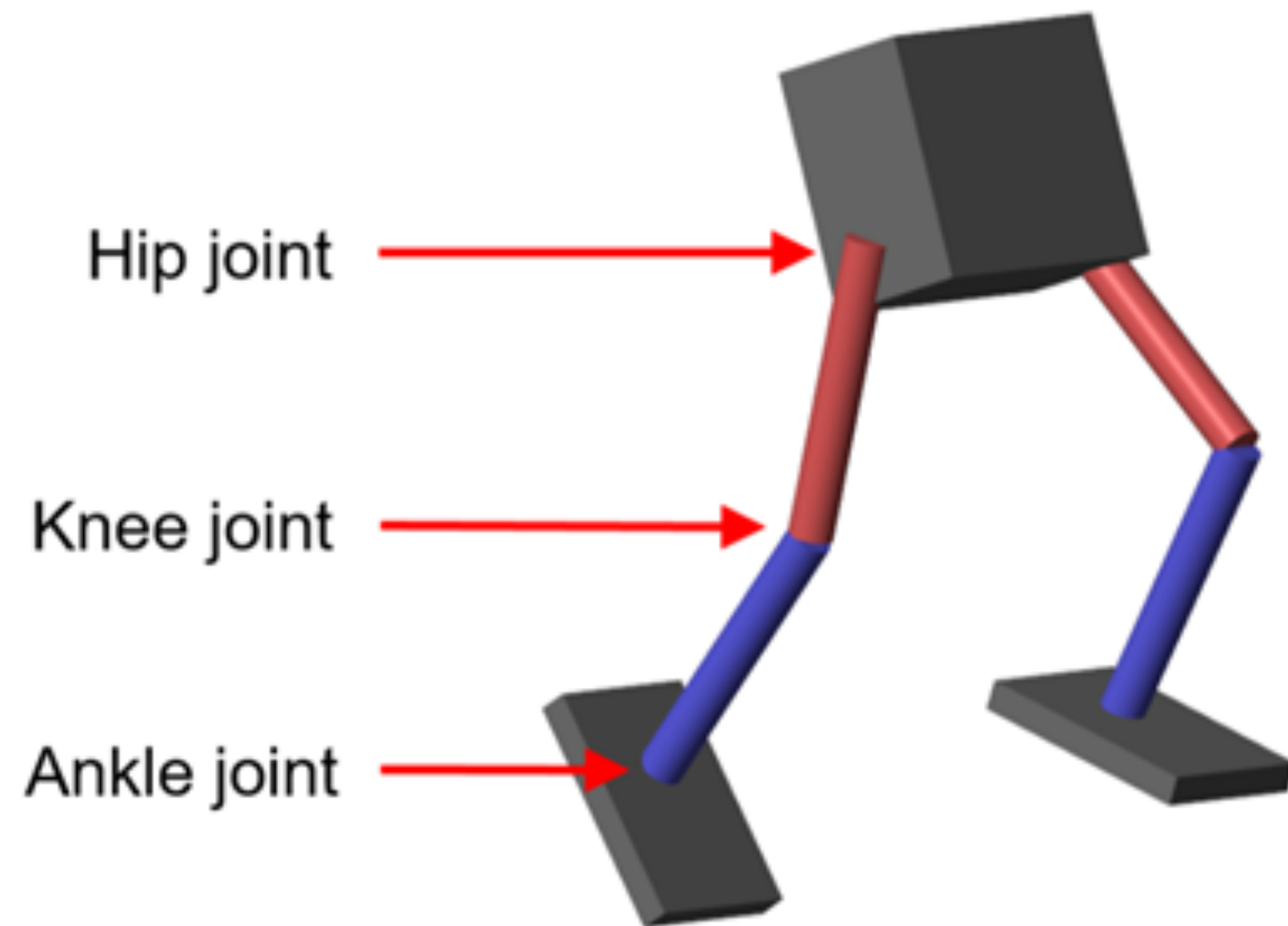
Reinforcement Learning

4

(DDPG) Deep Deterministic Policy Gradient Agent



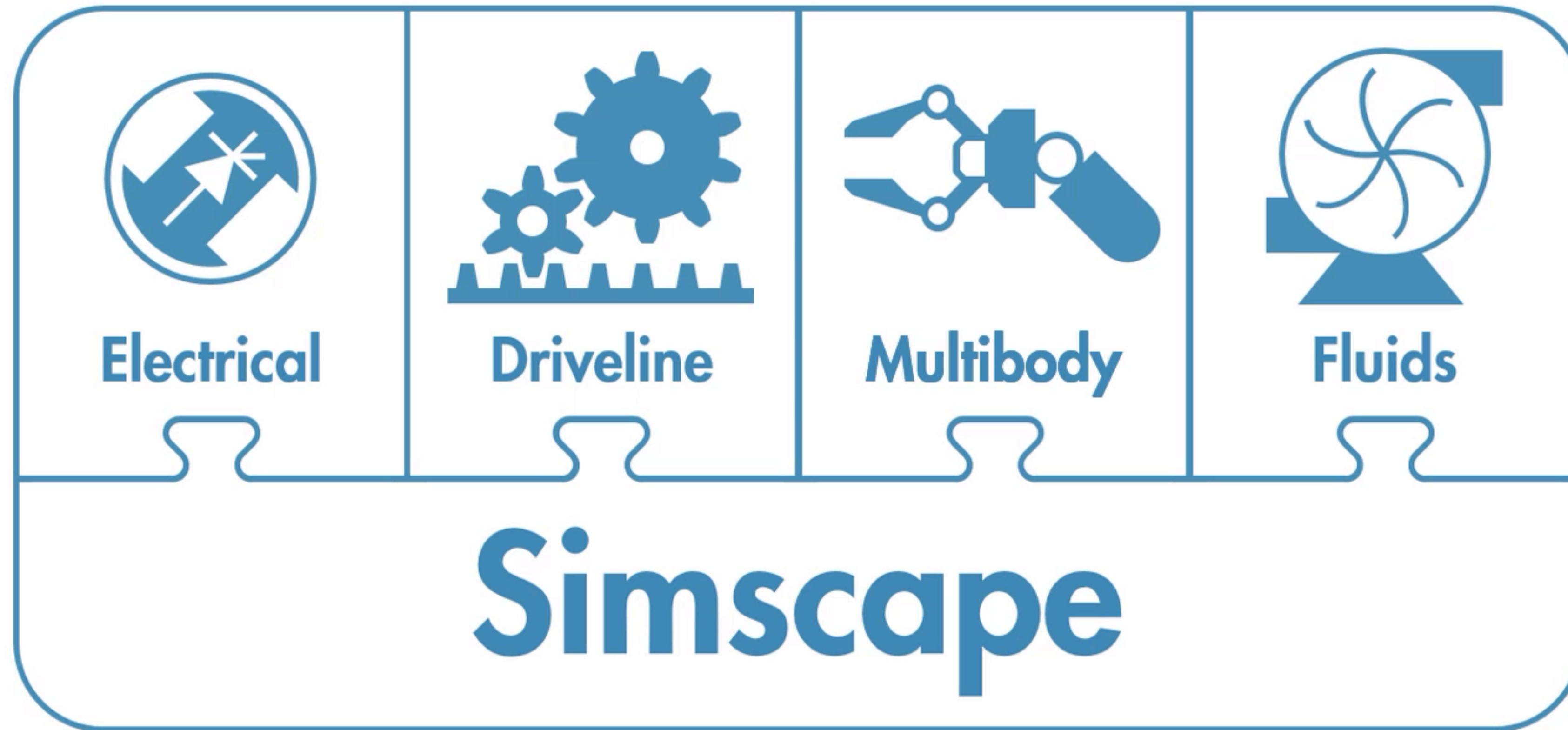
Environment



**How to model a
Biped Robot?**



Introduction to Simscape Multibody

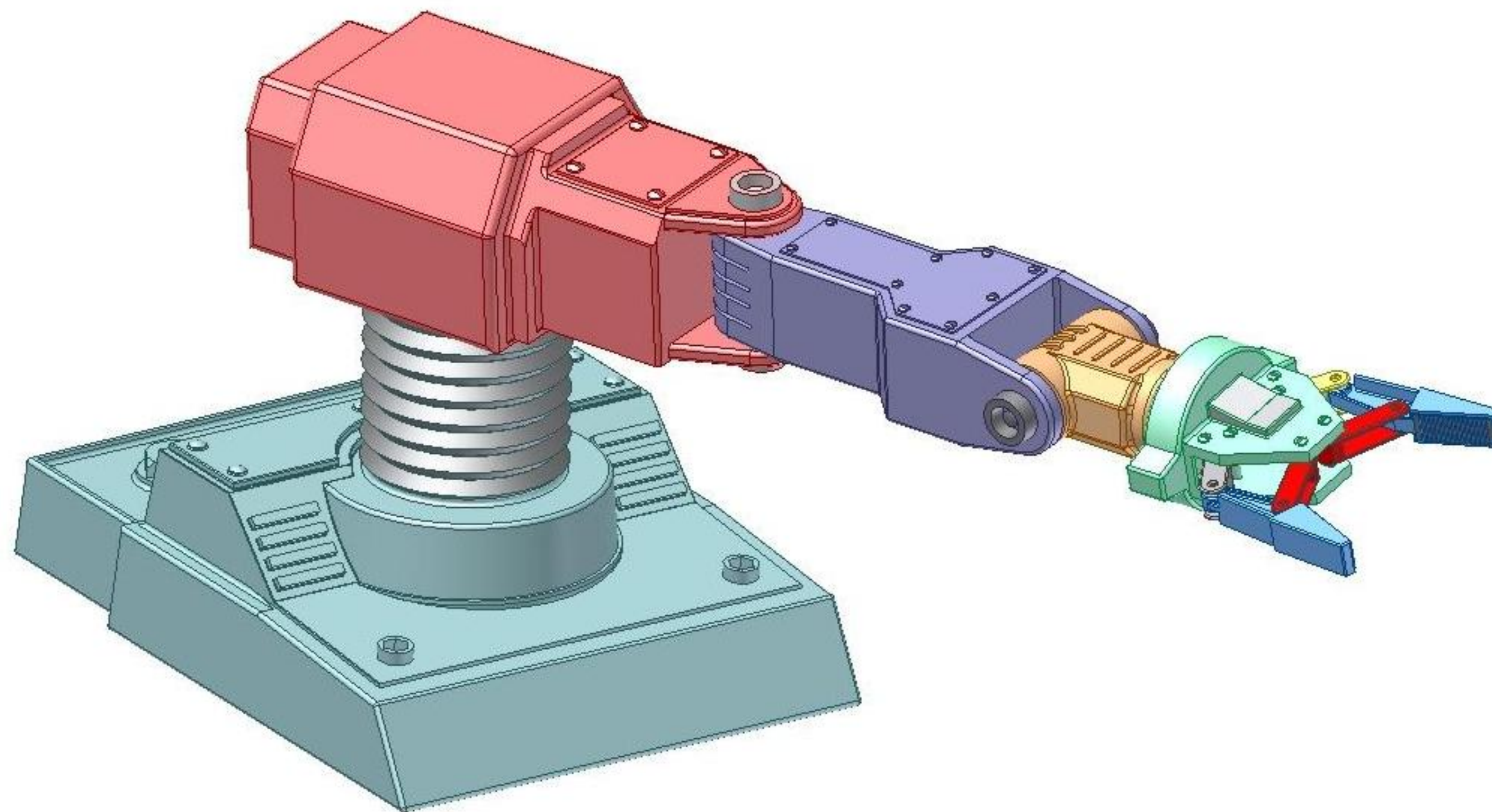


Short Video - Source: <https://nl.mathworks.com/products/simmechanics.html>



Environment

Option 1: Push of a button to import a CAD Model into Simulink



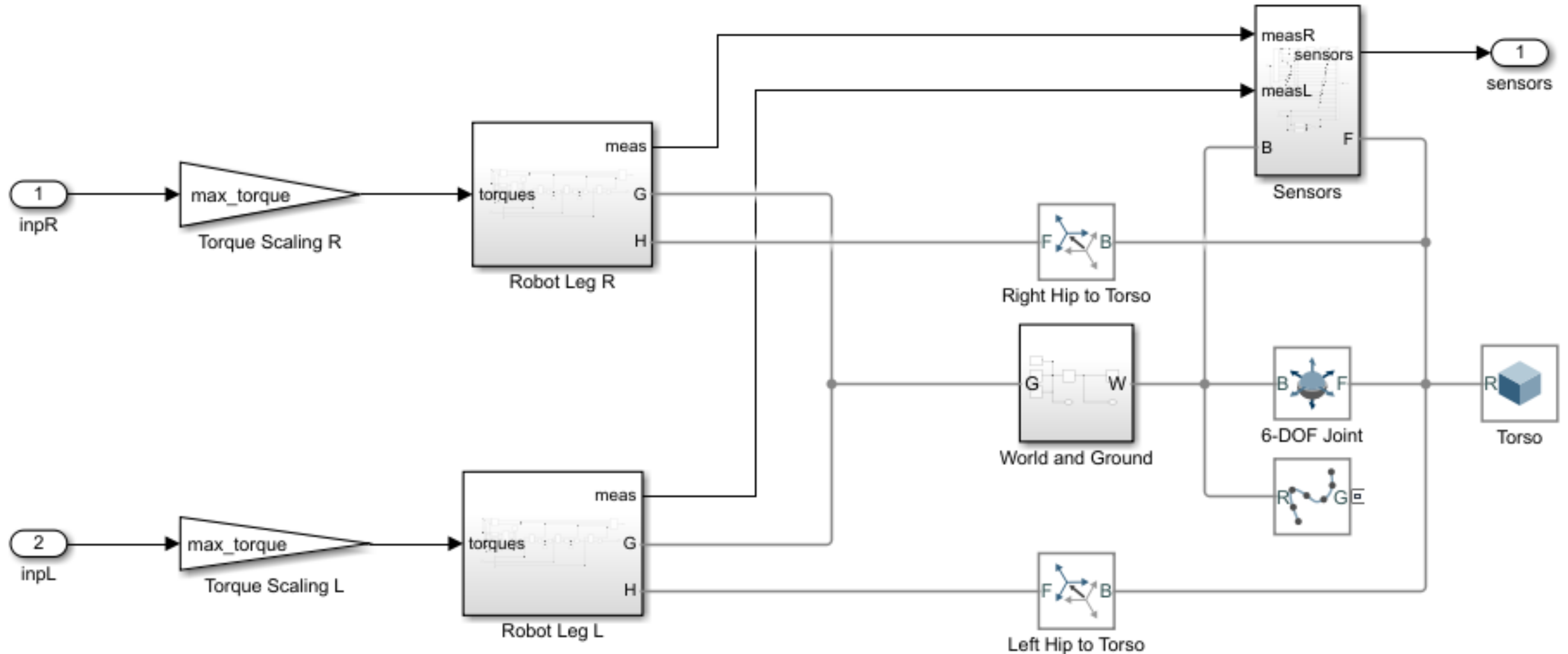
Simulink

Option 2: Make a simple Robot from MATLAB (easy first approach)
Explore Model

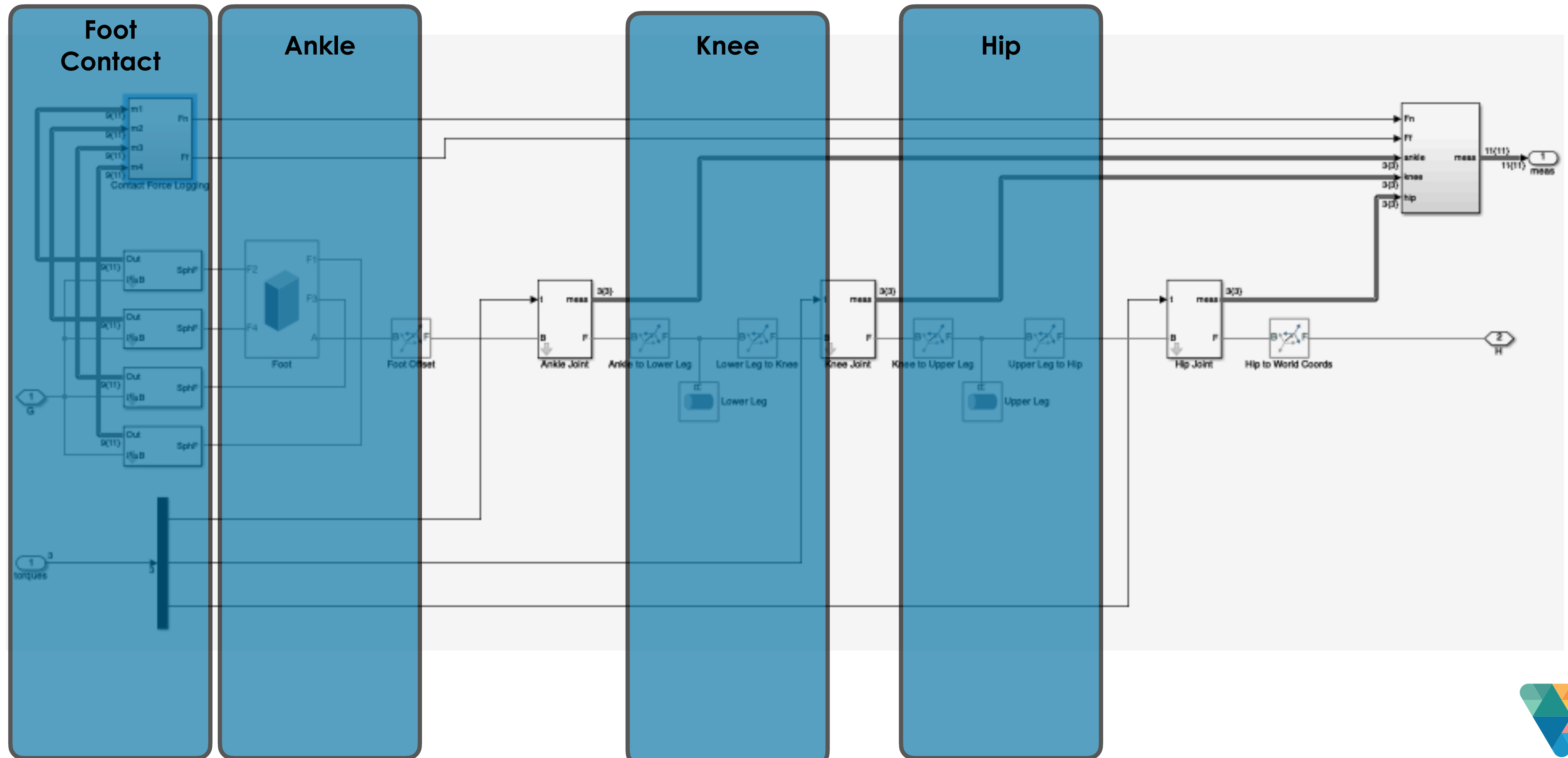


Biped Robot Model

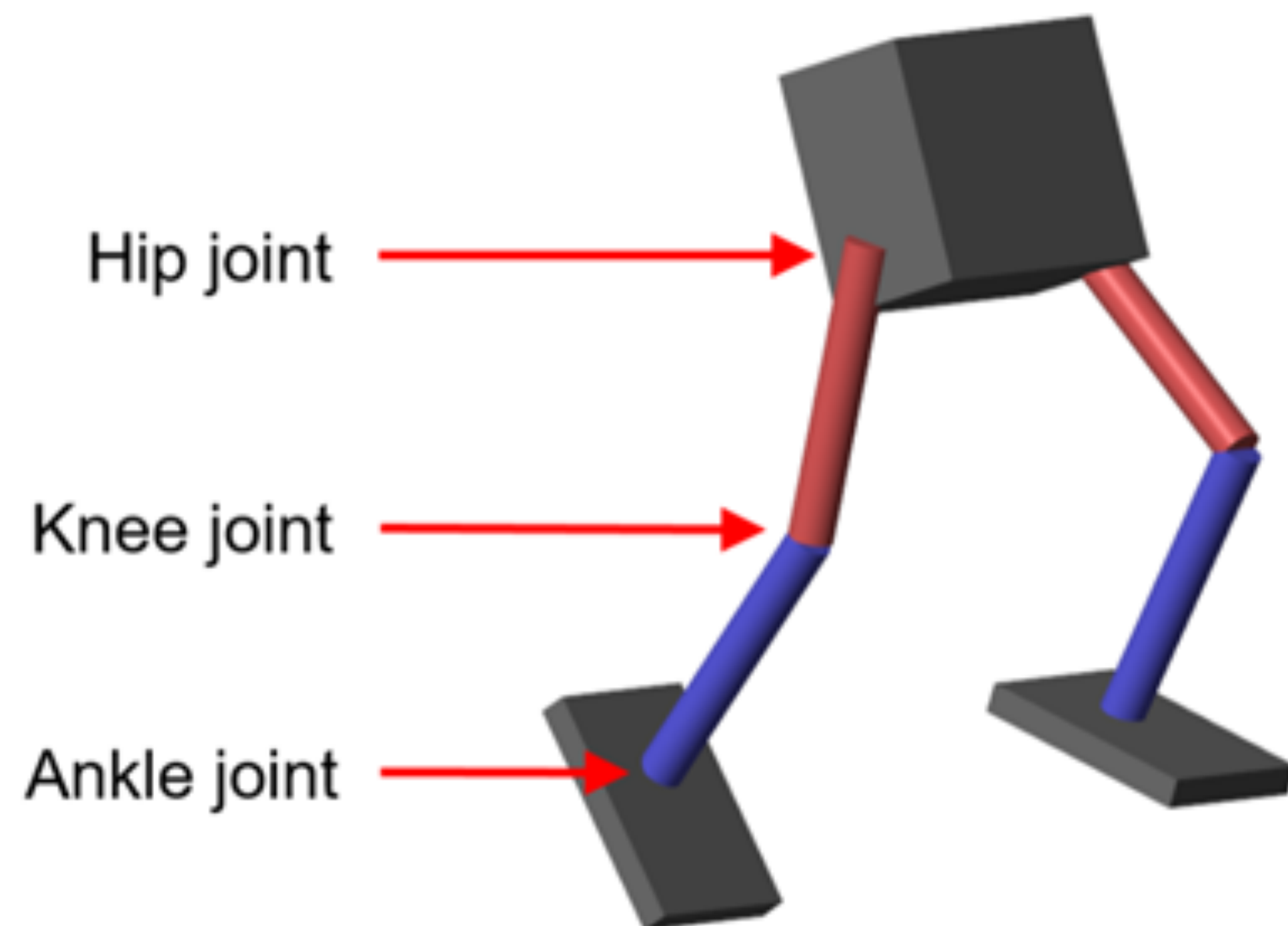
1



Biped Robot Model - Leg



Observations & Actions



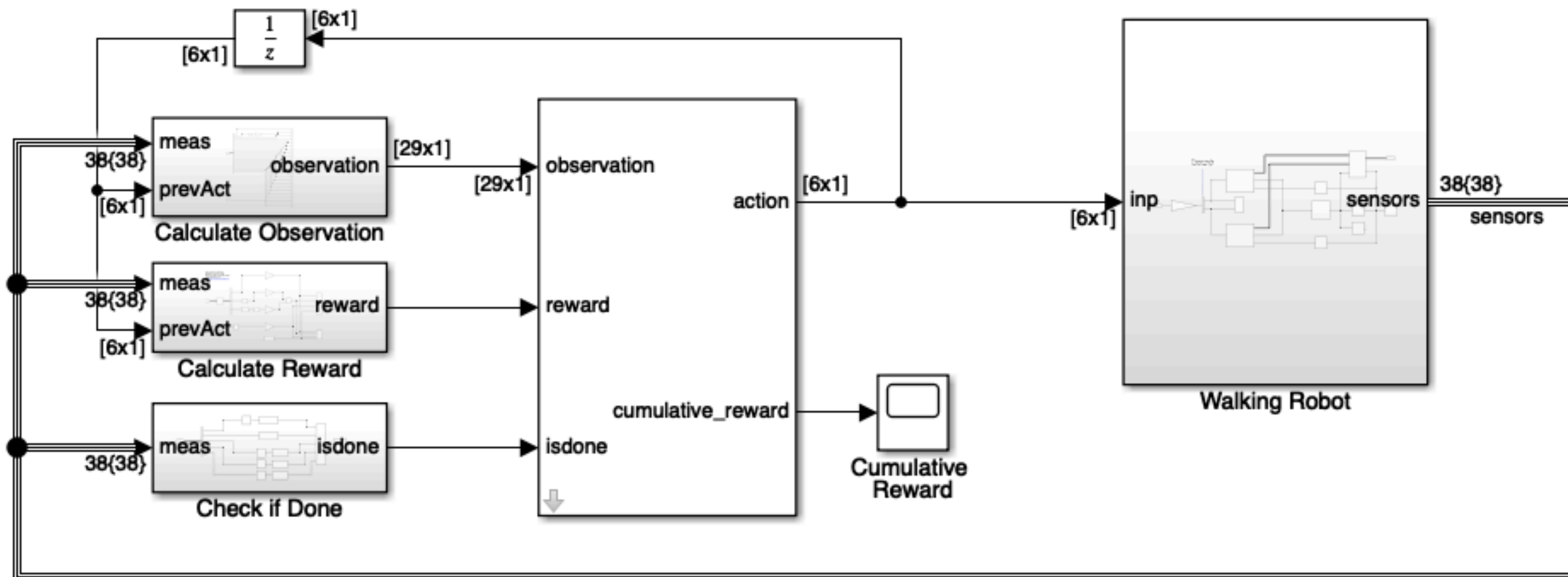
29 Observations
6 Actions



Observations & Actions

[Enable animation](#)

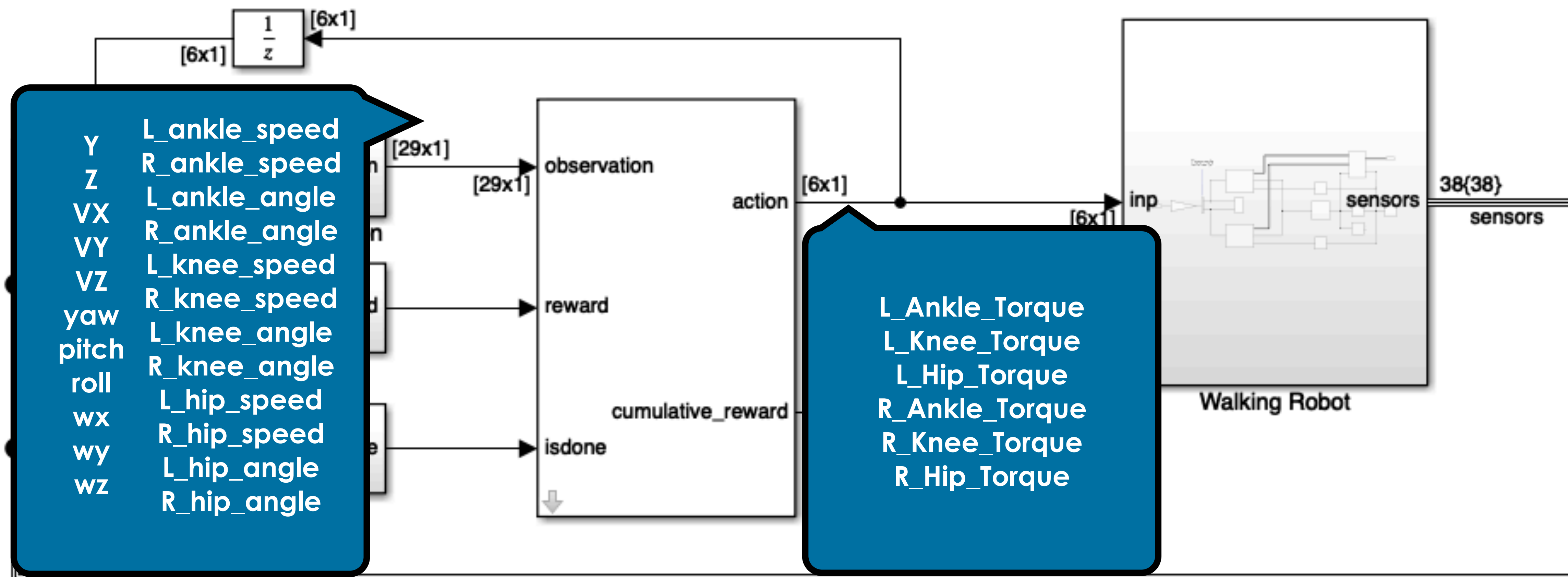
[Disable animation](#)



Observations & Actions

[Enable animation](#)

[Disable animation](#)



Reward - What is a good approach?

**Make a robot walk
in a
straight line
using minimal
control effort**



Reward

$$r_t = v_x - 3y^2 - 50\hat{z}^2 + 25\frac{T_s}{T_f} - 0.02\sum_i u_{t-1}^i{}^2$$

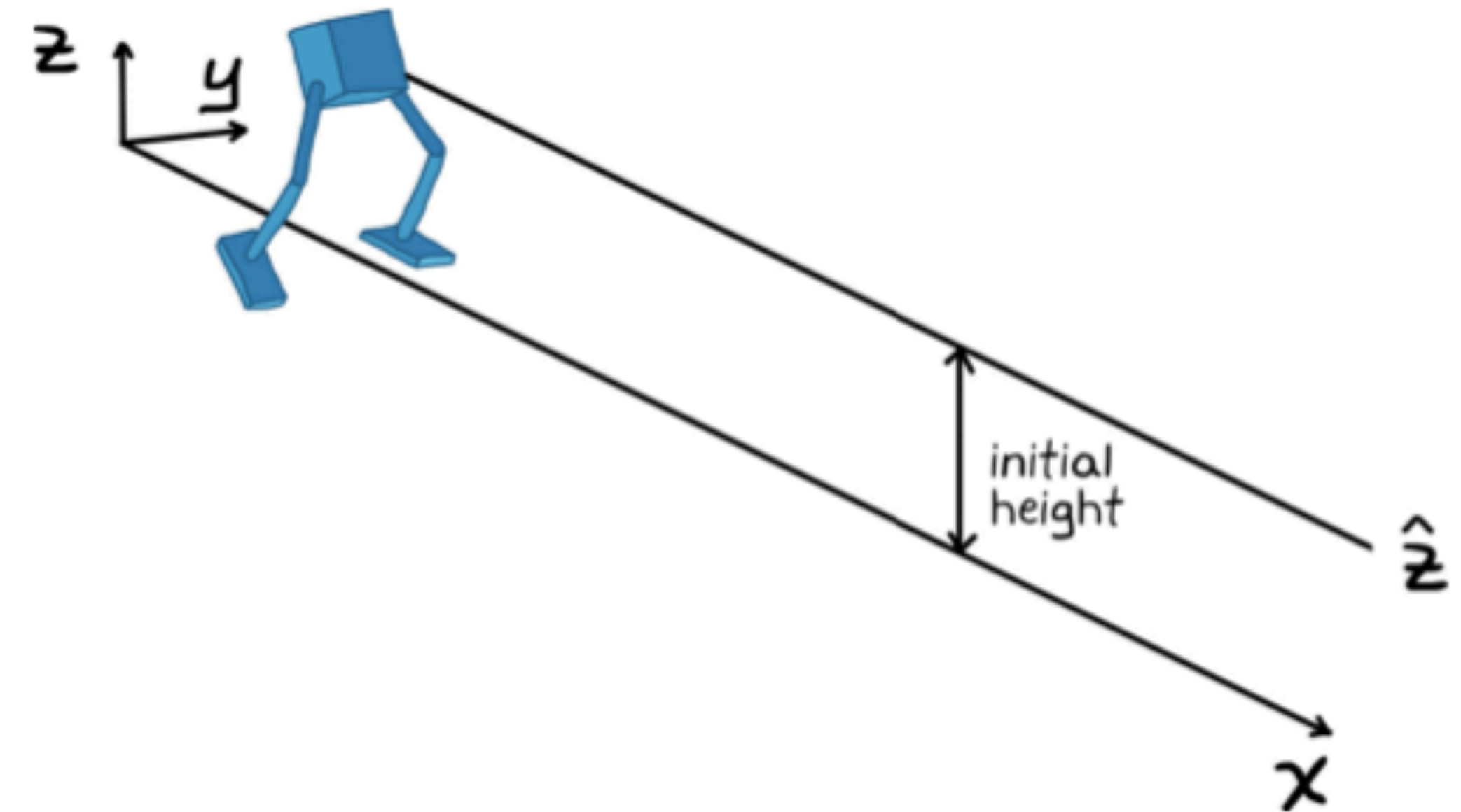
Forward Velocity

Don't Stray from the Path

Keep Trunk High

Walk as long as possible

Minimize Actuator Effort



Source: "Emergence of Locomotion Behaviours in Rich Environments"

Google DeepMind, 2017

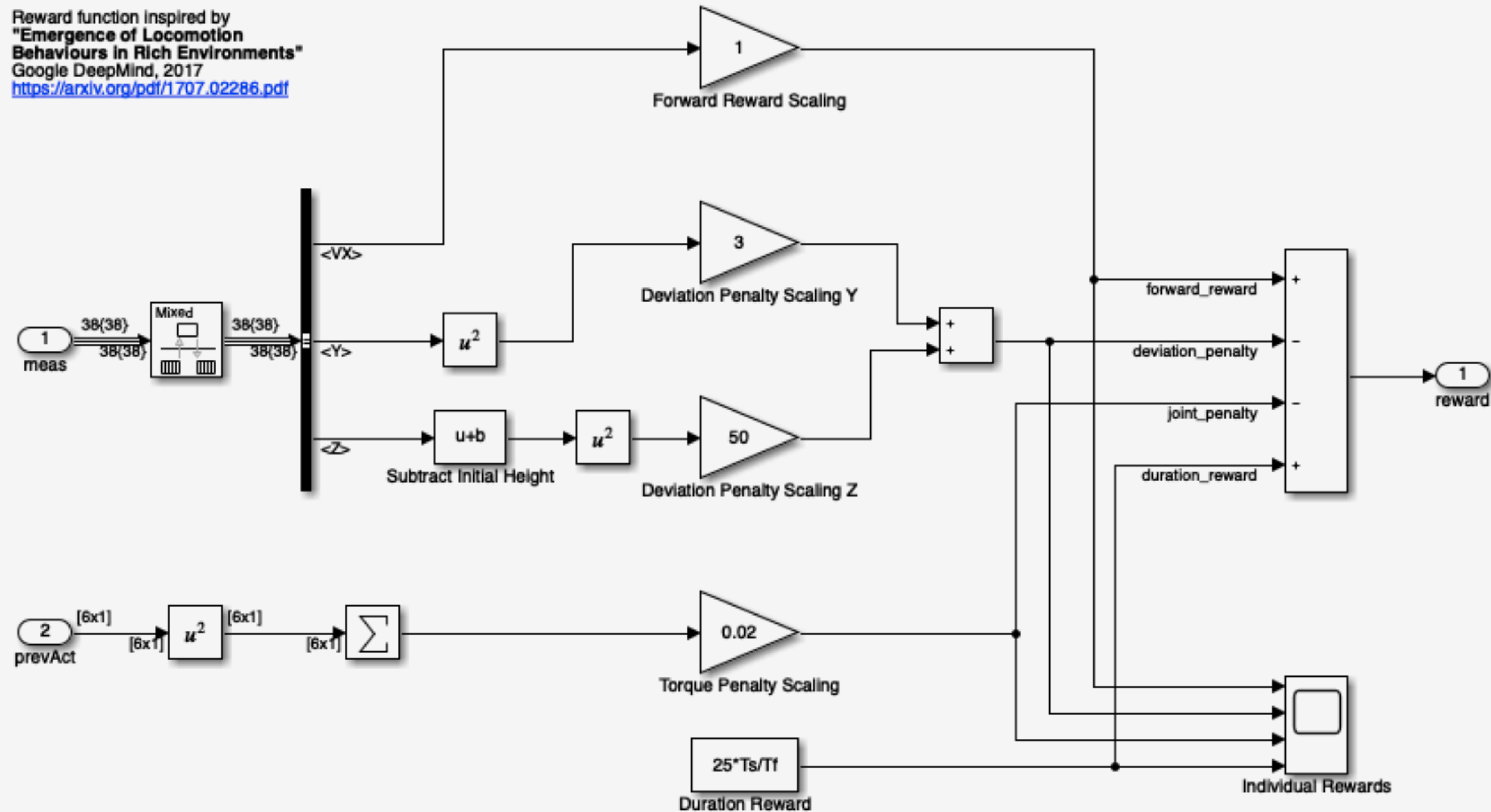
<https://arxiv.org/pdf/1707.02286.pdf>



Reward

$$r_t = v_x - 3y^2 - 50\hat{z}^2 + 25\frac{T_s}{T_f} - 0.02\sum_i u_{t-1}^2$$

Reward function inspired by
**"Emergence of Locomotion
 Behaviours in Rich Environments"**
 Google DeepMind, 2017
<https://arxiv.org/pdf/1707.02286.pdf>



Policy

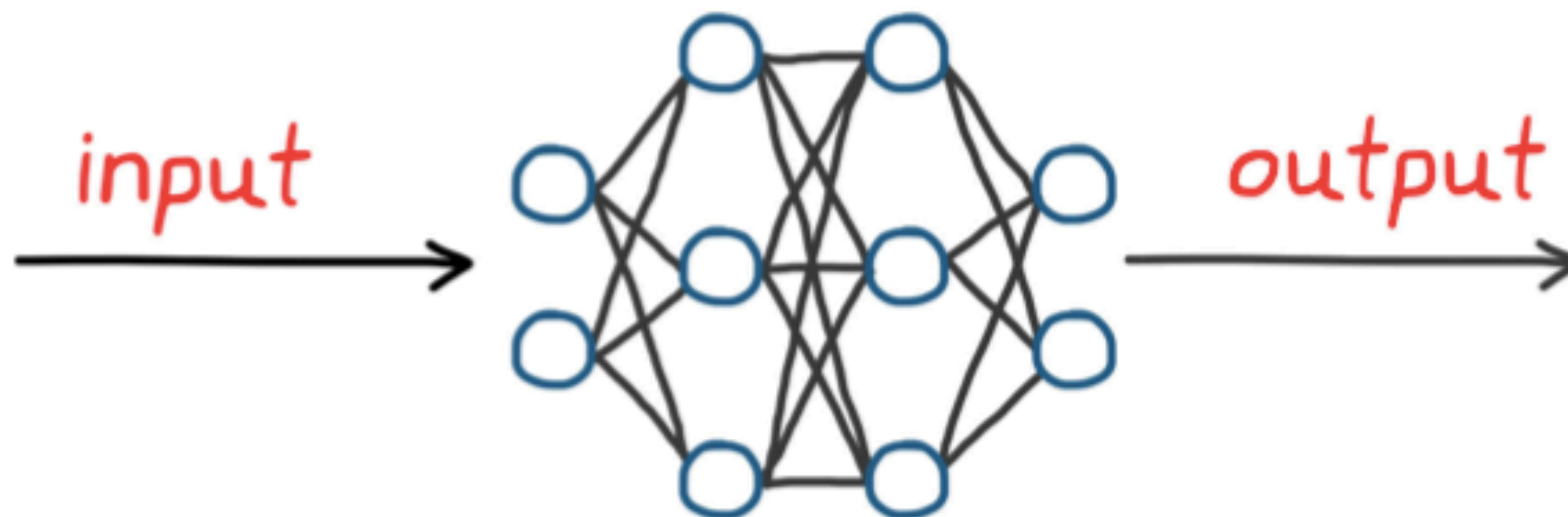
- Concepts
- Universal Function Approximate
- A Bit of Maths (for the non-data scientists)
- Neural Network Types
- Reinforcement Learning Algorithms



Policy

neural network

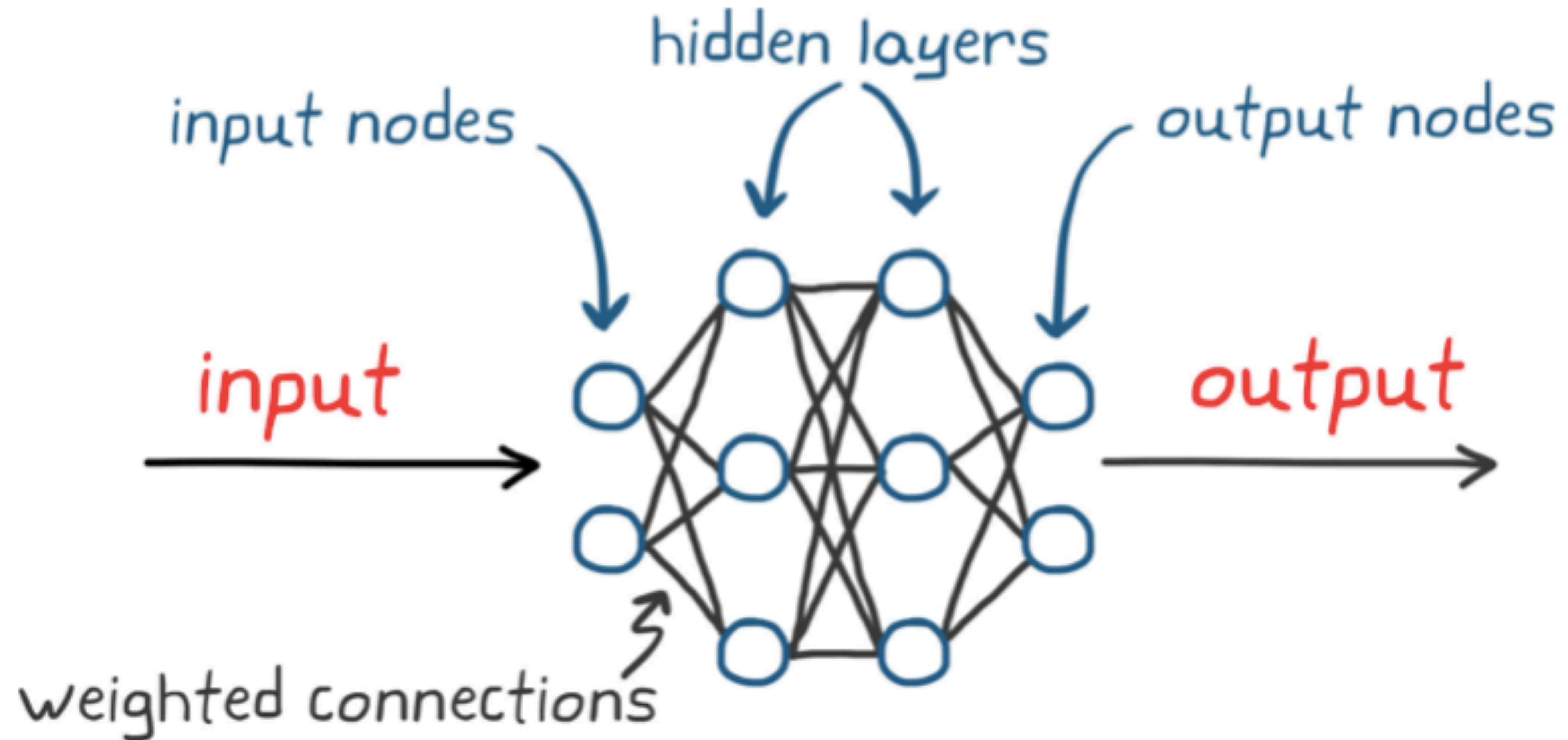
can mimic any input/output relationship



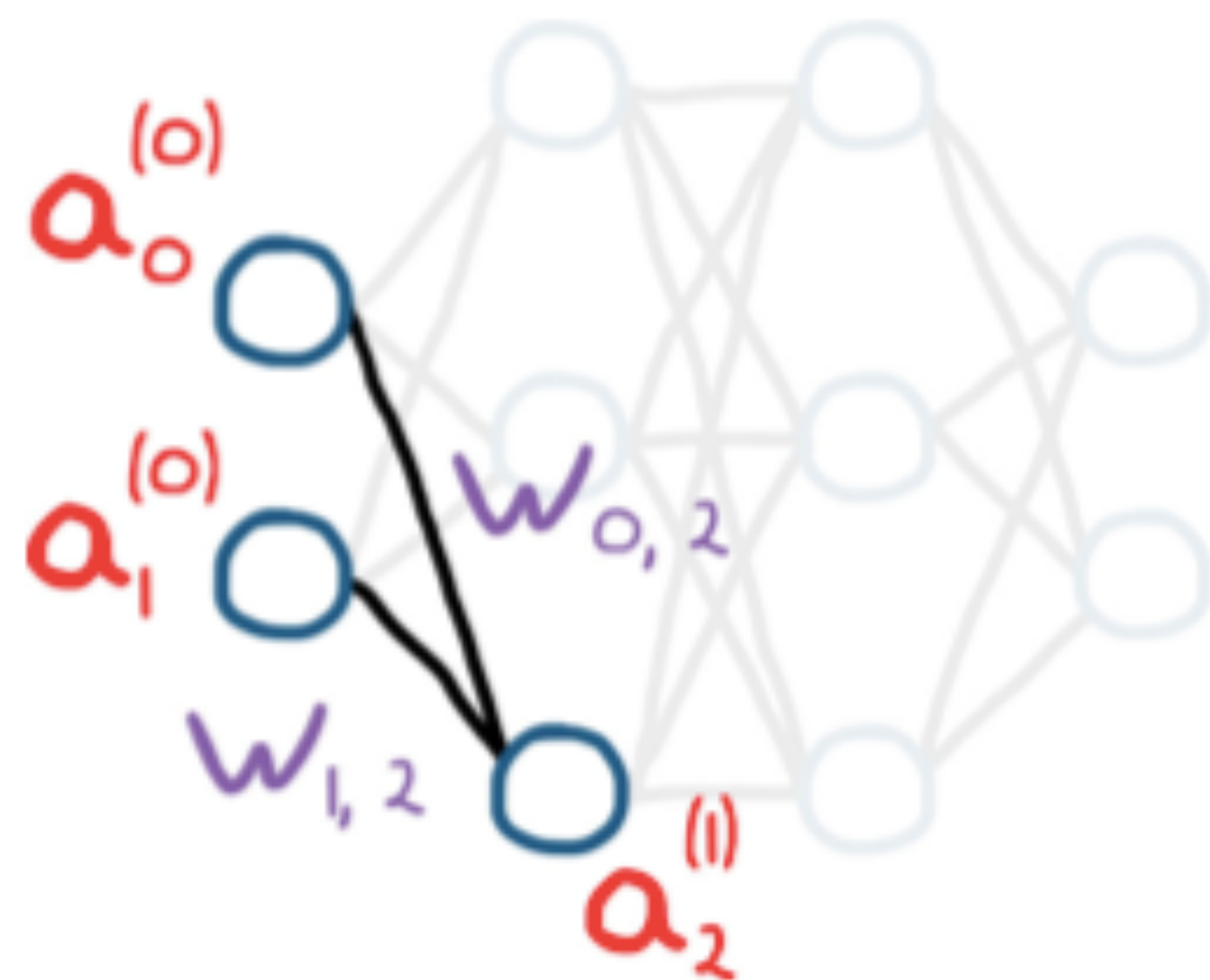
universal function approximator



Policy - A Bit of Maths



Policy - A Bit of Maths



value of $a_2^{(1)}$ = $w_{0,2} \cdot a_0^{(0)} + w_{1,2} \cdot a_1^{(0)} + b_2^{(1)}$

layer
node position



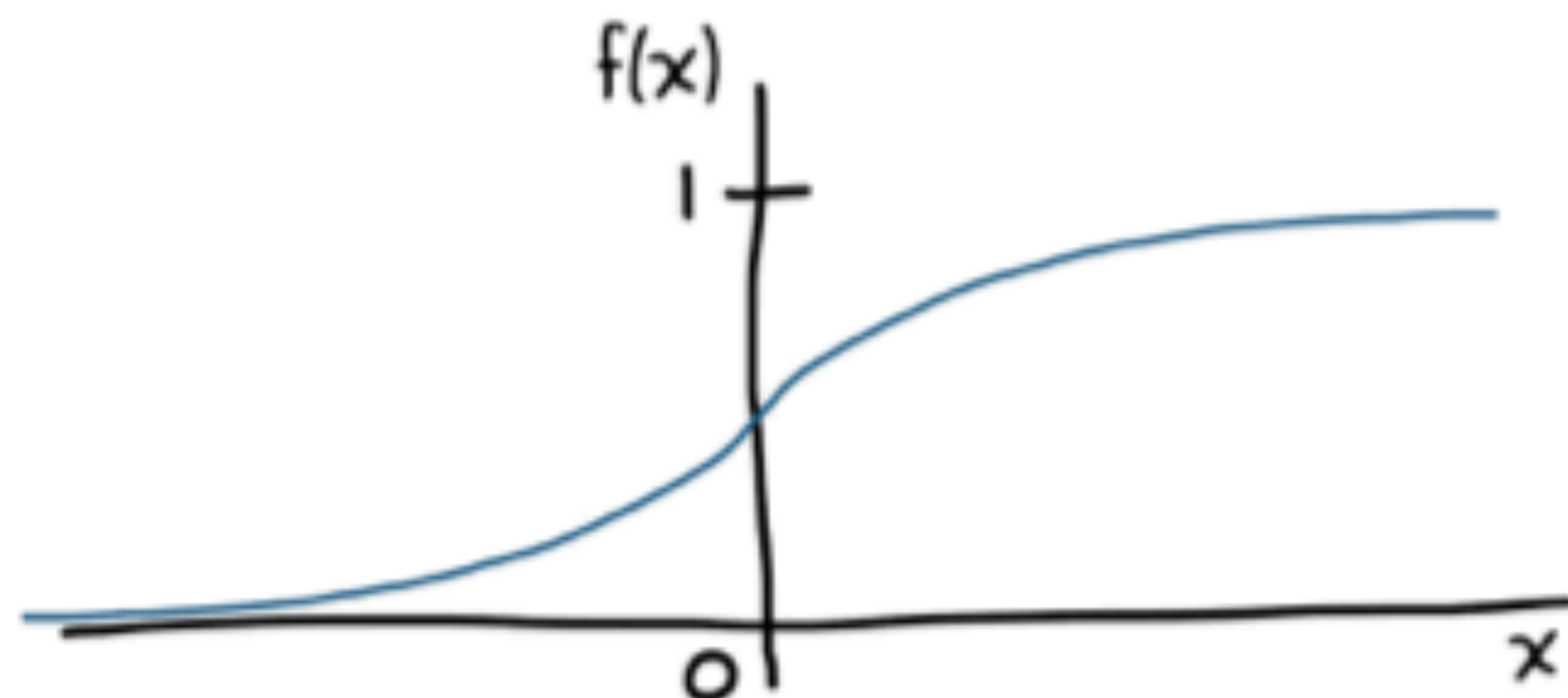
Policy - A Bit of Maths

$$a^{(1)} = \text{act} [w_0 a^{(0)} + b^{(1)}]$$

activation function is applied after linear operations

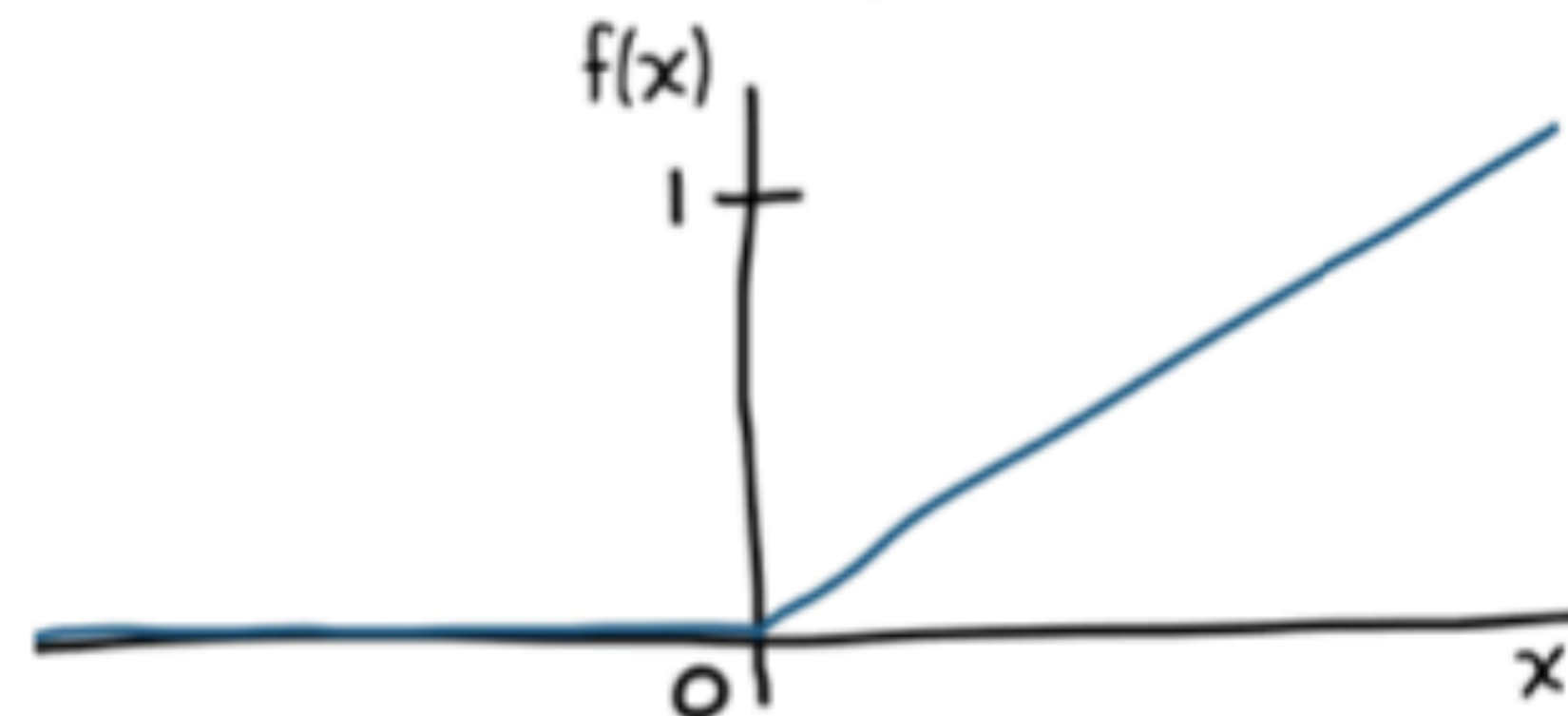
Sigmoid

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$



ReLU

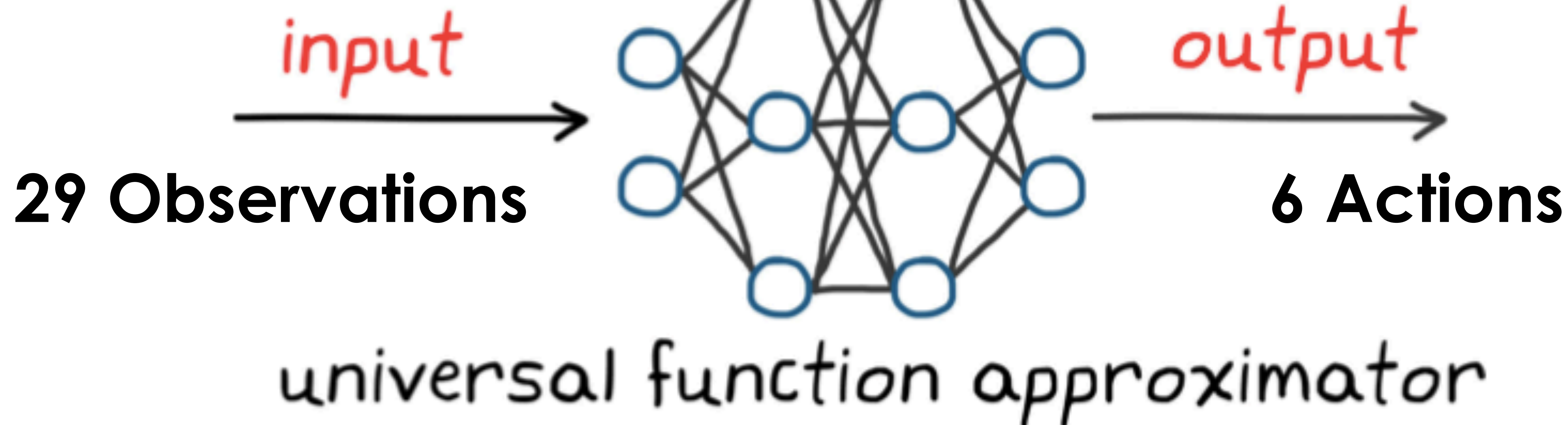
$$\begin{aligned} f(x) &= 0 \text{ for } x \leq 0 \\ f(x) &= x \text{ for } x > 0 \end{aligned}$$



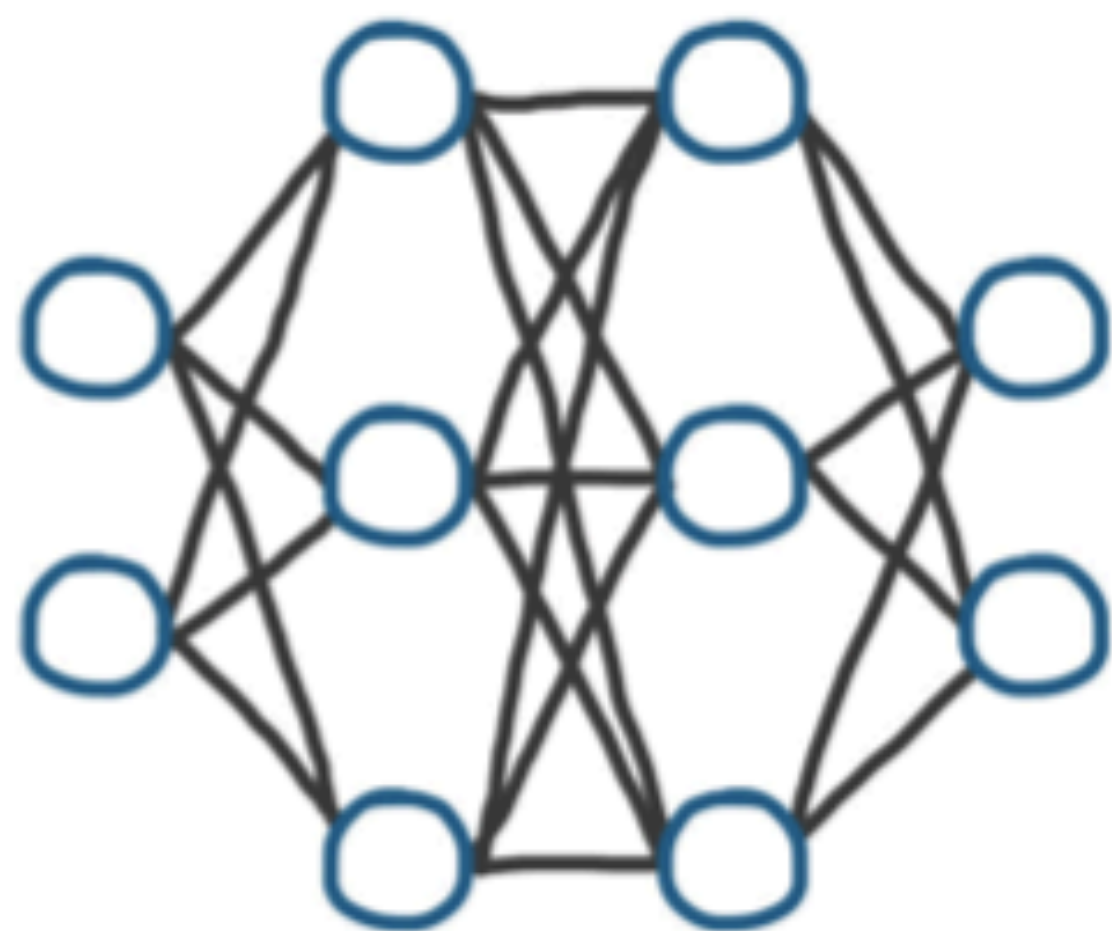
Policy

neural network

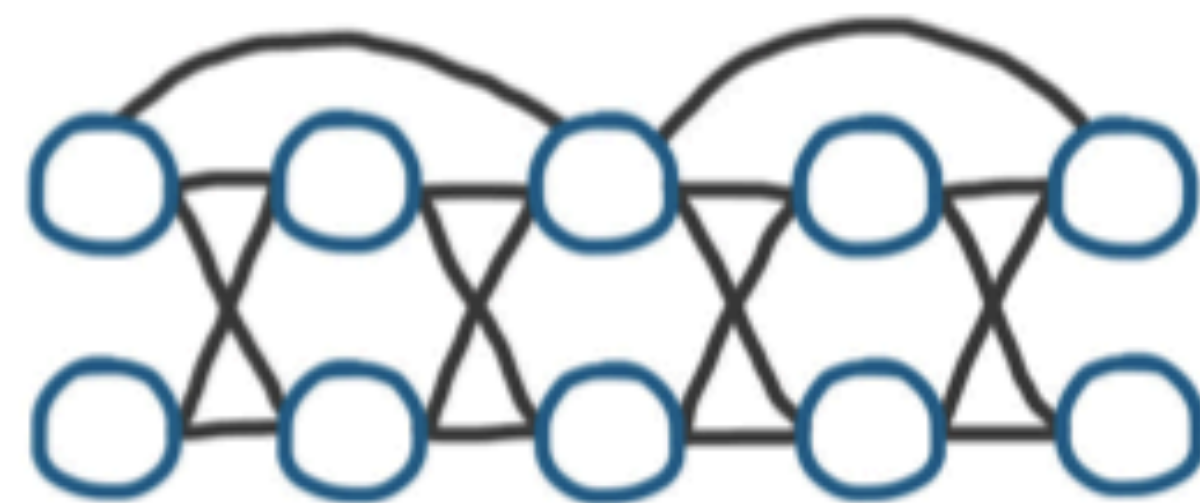
can mimic any input/output relationship



Policy - Neural Network Types



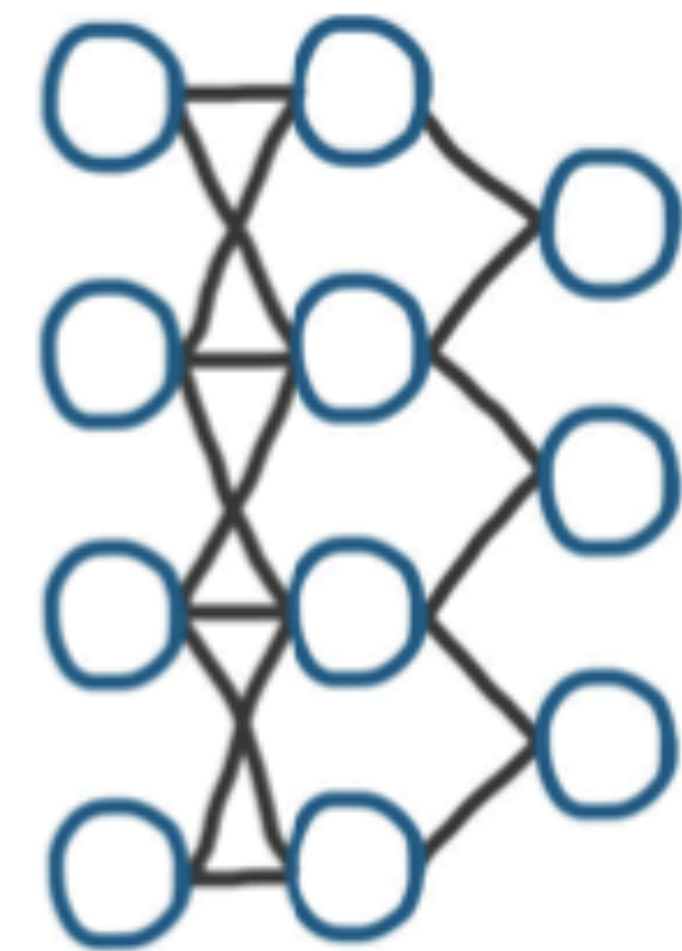
fully connected



residual



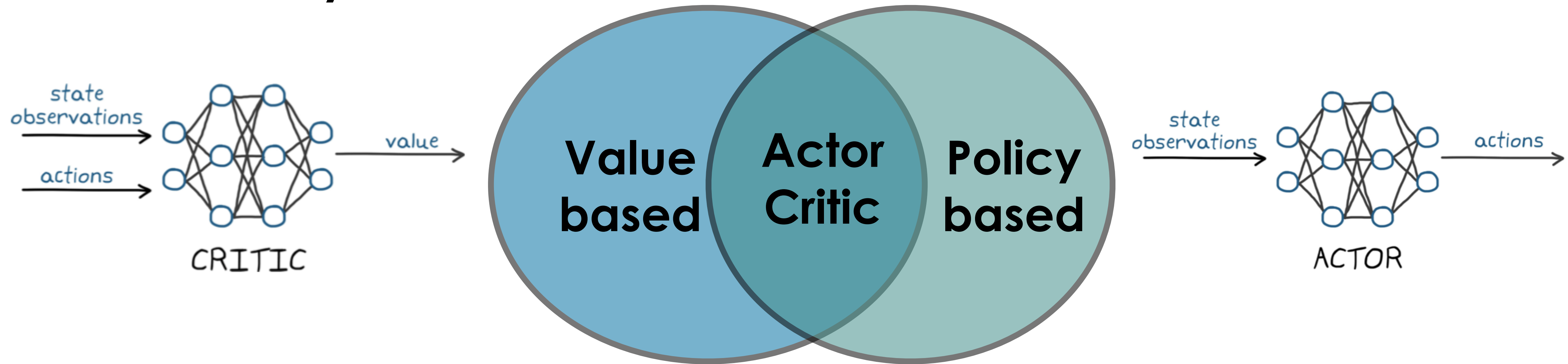
recurrent



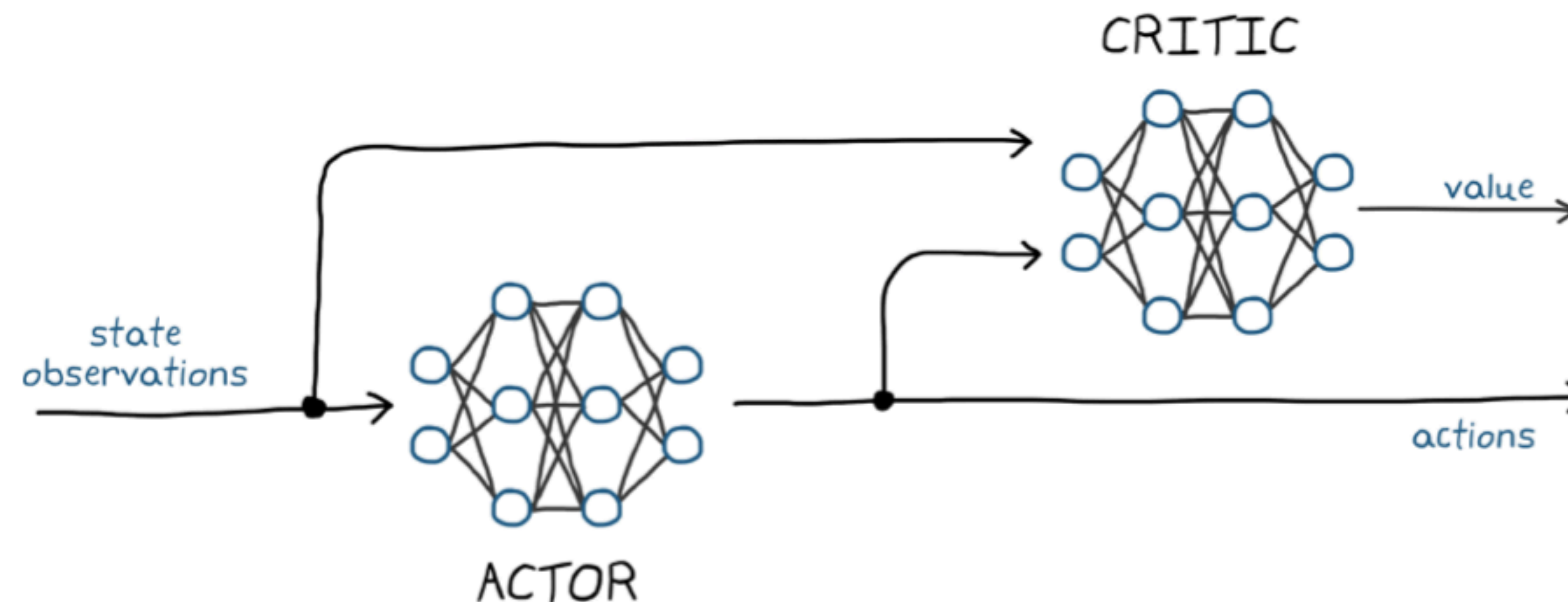
convolutional



Policy - Structures

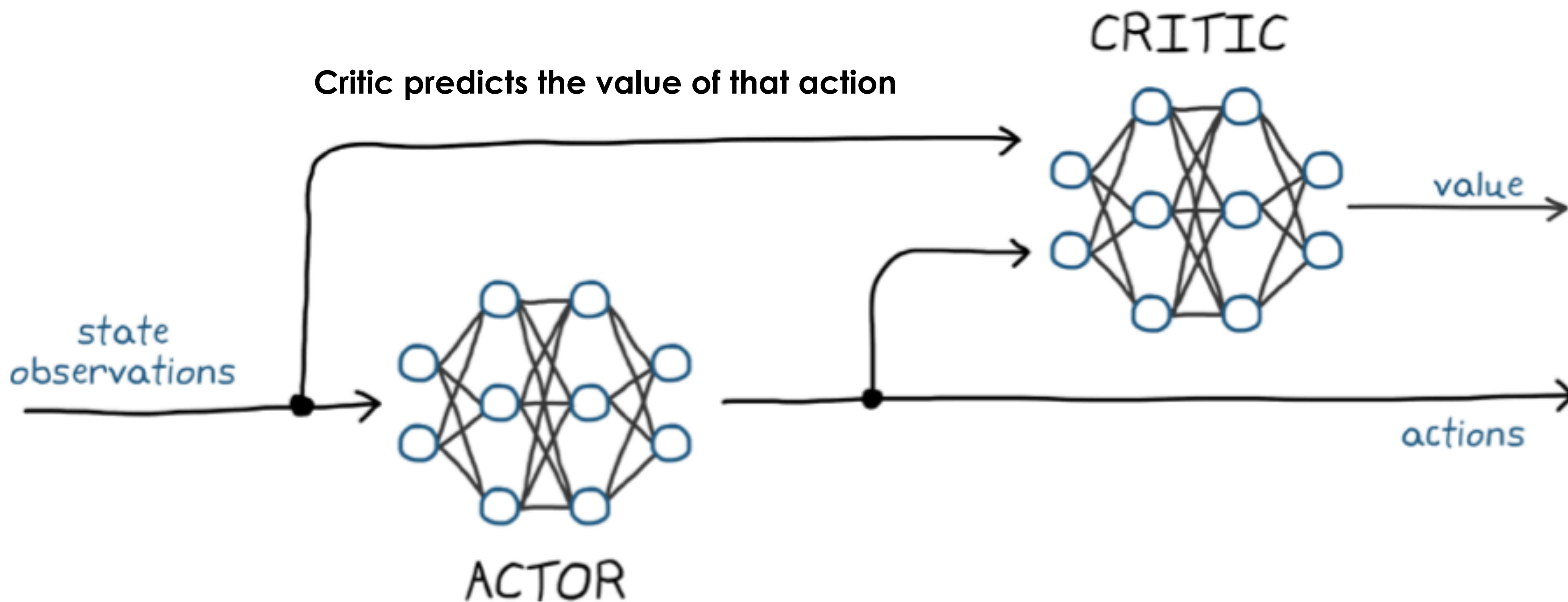


actor-critic learning algorithms



Policy - Actor-Critic

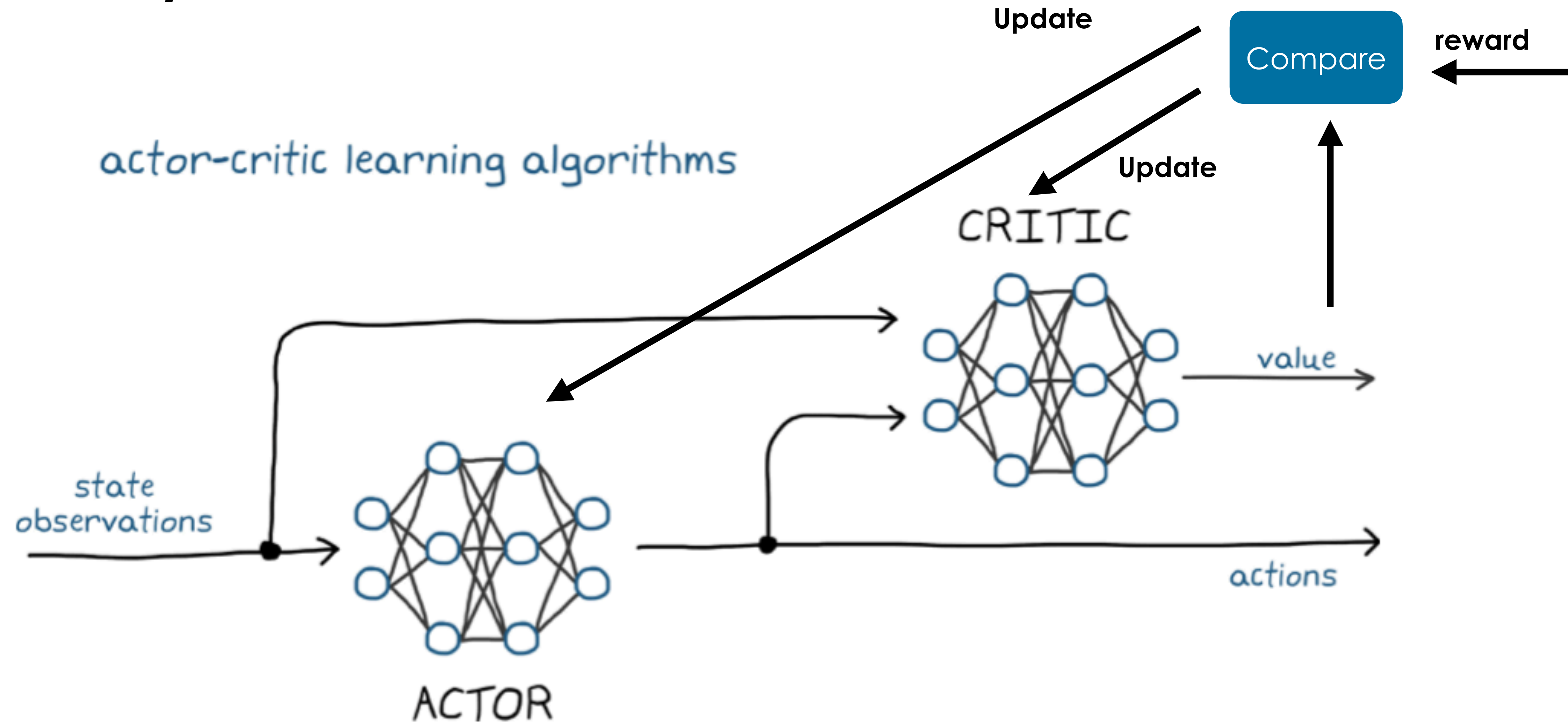
actor-critic learning algorithms



Responsible for Choosing the Actions



Policy - Actor-Critic



Policy

```
function [criticNetwork, actorNetwork] = createNetworks(numObs, numAct)
% Walking Robot -- DDPG Neural Network Setup Script
% Copyright 2019 The MathWorks, Inc.
```

```
% Network structure inspired by original 2015 DDPG paper
% "Continuous Control with Deep Reinforcement Learning", Lillicrap et al.
% https://arxiv.org/pdf/1509.02971.pdf
```

```
%% CRITIC
% Create the critic network layers
criticLayerSizes = [400 300];
statePath = [
    imageInputLayer([numObs 1 1], 'Normalization', 'none', 'Name', 'observation')
    fullyConnectedLayer(criticLayerSizes(1), 'Name', 'CriticStateFC1', ...
        'Weights', 2/sqrt(numObs)*(rand(criticLayerSizes(1), numObs)-0.5), ...
        'Bias', 2/sqrt(numObs)*(rand(criticLayerSizes(1), 1)-0.5))
    reluLayer('Name', 'CriticStateRelu1')
    fullyConnectedLayer(criticLayerSizes(2), 'Name', 'CriticStateFC2', ...
        'Weights', 2/sqrt(criticLayerSizes(1))*(rand(criticLayerSizes(2), criticLayerSizes(1))-0.5), ...
        'Bias', 2/sqrt(criticLayerSizes(1))*(rand(criticLayerSizes(2), 1)-0.5))
];
actionPath = [
    imageInputLayer([numAct 1 1], 'Normalization', 'none', 'Name', 'action')
    fullyConnectedLayer(criticLayerSizes(2), 'Name', 'CriticActionFC1', ...
        'Weights', 2/sqrt(numAct)*(rand(criticLayerSizes(2), numAct)-0.5), ...
        'Bias', 2/sqrt(numAct)*(rand(criticLayerSizes(2), 1)-0.5))
]; % Connect the layer graph
```

```
commonPath = [
    additionLayer(2, 'Name', 'add')
    reluLayer('Name', 'CriticCommonRelu1')
    fullyConnectedLayer(1, 'Name', 'CriticOutput', ...
        'Weights', 2*5e-3*(rand(1, criticLayerSizes(2))-0.5), ...
        'Bias', 2*5e-3*(rand(1, 1)-0.5))
];
```

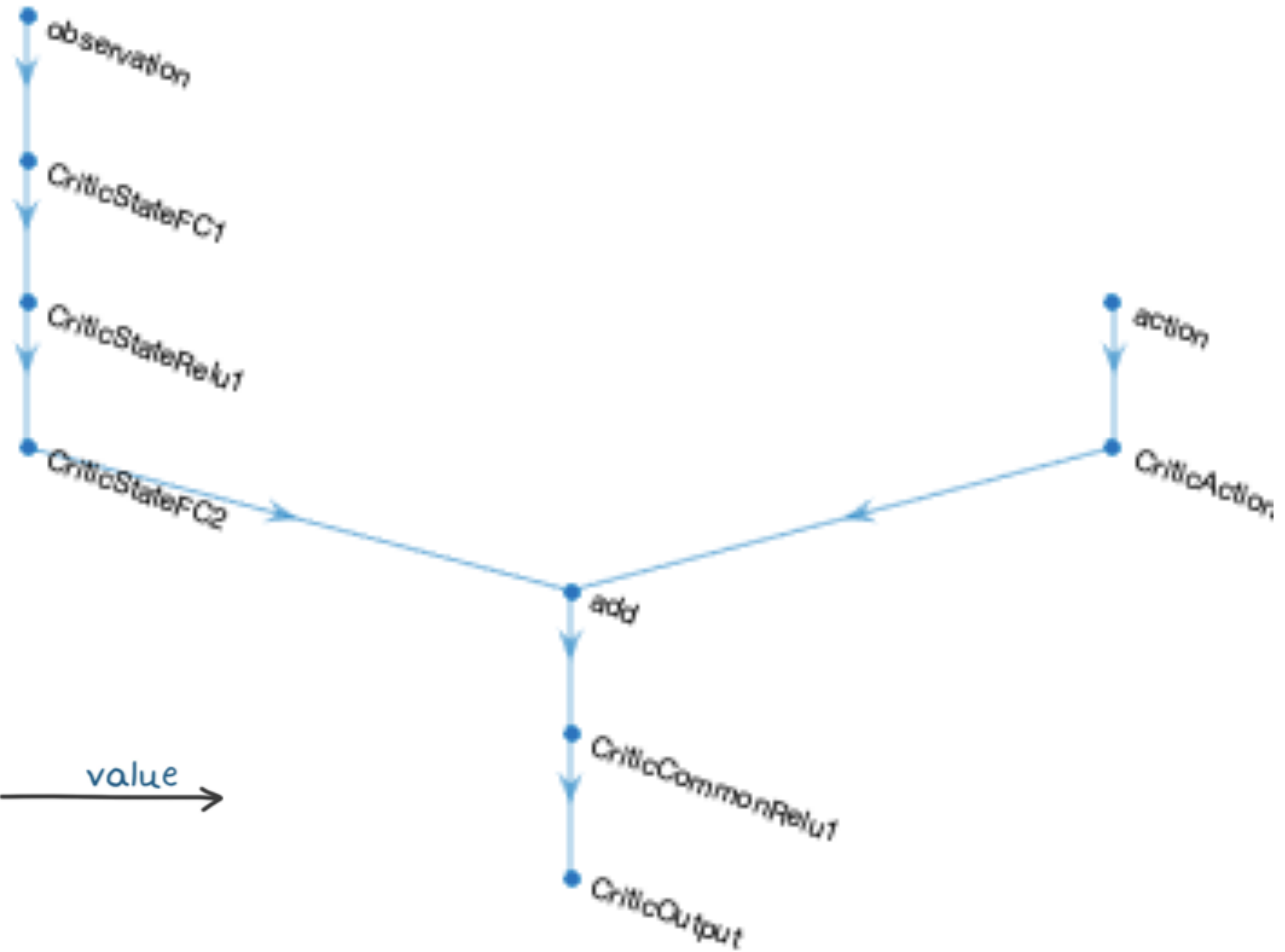
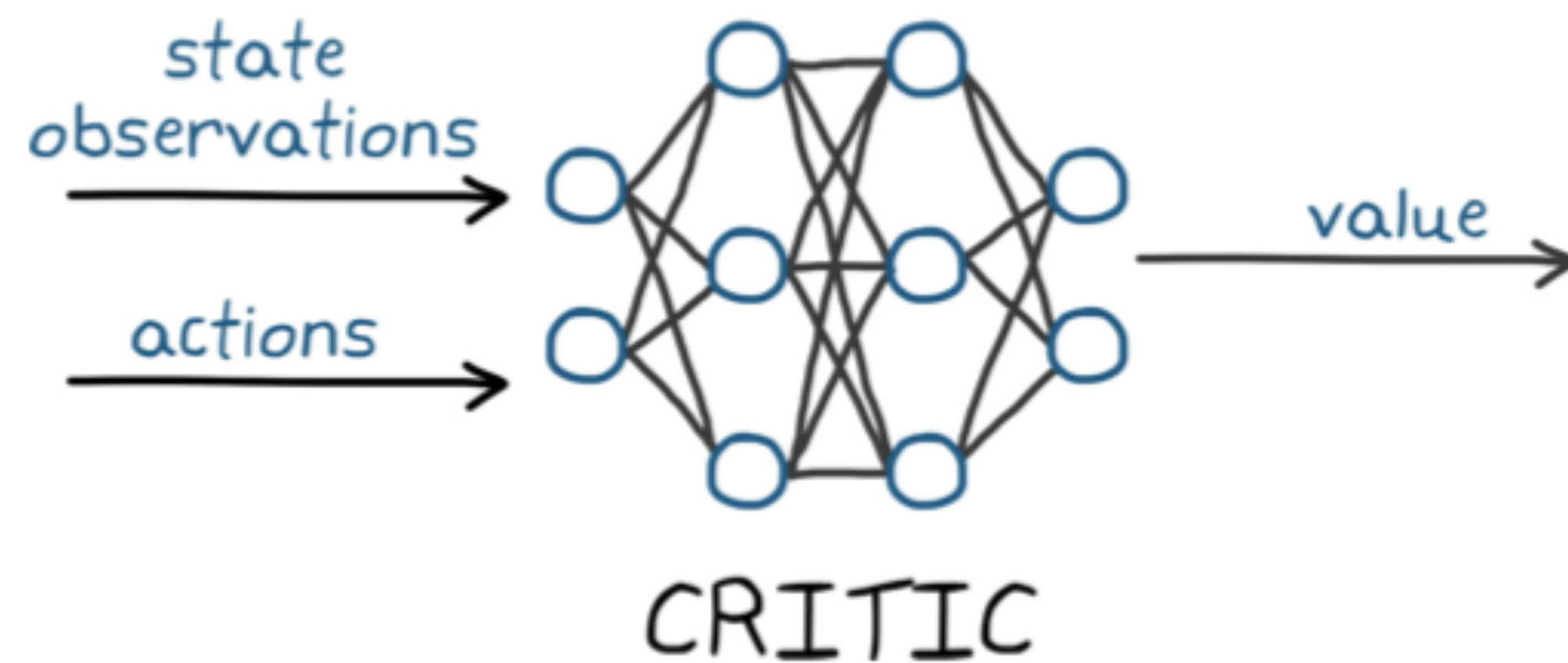
```
% Connect the layer graph
criticNetwork = layerGraph(statePath);
criticNetwork = addLayers(criticNetwork, actionPath);
criticNetwork = addLayers(criticNetwork, commonPath);
criticNetwork = connectLayers(criticNetwork, 'CriticStateFC2', 'add/in1');
criticNetwork = connectLayers(criticNetwork, 'CriticActionFC1', 'add/in2');
```

```
%% ACTOR
% Create the actor network layers
actorLayerSizes = [400 300];
actorNetwork = [
    imageInputLayer([numObs 1 1], 'Normalization', 'none', 'Name', 'observation')
    fullyConnectedLayer(actorLayerSizes(1), 'Name', 'ActorFC1', ...
        'Weights', 2/sqrt(numObs)*(rand(actorLayerSizes(1), numObs)-0.5), ...
        'Bias', 2/sqrt(numObs)*(rand(actorLayerSizes(1), 1)-0.5))
    reluLayer('Name', 'ActorRelu1')
    fullyConnectedLayer(actorLayerSizes(2), 'Name', 'ActorFC2', ...
        'Weights', 2/sqrt(actorLayerSizes(1))*(rand(actorLayerSizes(2), actorLayerSizes(1))-0.5), ...
        'Bias', 2/sqrt(actorLayerSizes(1))*(rand(actorLayerSizes(2), 1)-0.5))
    reluLayer('Name', 'ActorRelu2')
    fullyConnectedLayer(numAct, 'Name', 'ActorFC3', ...
        'Weights', 2*5e-3*(rand(numAct, actorLayerSizes(2))-0.5), ...
        'Bias', 2*5e-3*(rand(numAct, 1)-0.5))
    tanhLayer('Name', 'ActorTanh1')
];
```



Policy

3



Reinforcement Algorithms

Comparison of reinforcement learning algorithms [[edit](#)]

Algorithm	Description	Model	Policy	Action Space	State Space	Operator
Monte Carlo	Every visit to Monte Carlo	Model-Free	Off-policy	Discrete	Discrete	Sample-means
Q-learning	State–action–reward–state	Model-Free	Off-policy	Discrete	Discrete	Q-value
SARSA	State–action–reward–state–action	Model-Free	On-policy	Discrete	Discrete	Q-value
Q-learning - Lambda	State–action–reward–state with eligibility traces	Model-Free	Off-policy	Discrete	Discrete	Q-value
SARSA - Lambda	State–action–reward–state–action with eligibility traces	Model-Free	On-policy	Discrete	Discrete	Q-value
DQN	Deep Q Network	Model-Free	Off-policy	Discrete	Continuous	Q-value
DDPG	Deep Deterministic Policy Gradient	Model-Free	Off-policy	Continuous	Continuous	Q-value
A3C	Asynchronous Advantage Actor-Critic Algorithm	Model-Free	On-policy	Continuous	Continuous	Advantage
NAF	Q-Learning with Normalized Advantage Functions	Model-Free	Off-policy	Continuous	Continuous	Advantage
TRPO	Trust Region Policy Optimization	Model-Free	On-policy	Continuous	Continuous	Advantage
PPO	Proximal Policy Optimization	Model-Free	On-policy	Continuous	Continuous	Advantage
TD3	Twin Delayed Deep Deterministic Policy Gradient	Model-Free	Off-policy	Continuous	Continuous	Q-value
SAC	Soft Actor-Critic	Model-Free	Off-policy	Continuous	Continuous	Advantage

Source: https://en.wikipedia.org/wiki/Reinforcement_learning



DDPG - Deep Deterministic Policy Gradient

Model-Free, online, off-policy, reinforcement learning method.

DDPG is an actor-critic reinforcement learning agent that computes optimal policy that maximizes the long-term reward.

Additional Information:

<https://nl.mathworks.com/help/reinforcement-learning/ug/ddpg-agents.html>

<https://arxiv.org/pdf/1509.02971.pdf>



DDPG - Deep Deterministic Policy Gradient

Create the critic and actor representations using the specified deep neural networks and options. You must also specify the action and observation information for each representation, which you already obtained from the environment interface. For more information, see [rlRepresentation](#).

```
critic = rlRepresentation(criticNetwork,obsInfo,actInfo,'Observation',{'observation'},'Action',{'action'},criticOptions);  
actor  = rlRepresentation(actorNetwork,obsInfo,actInfo,'Observation',{'observation'},'Action',{'ActorTanh1'},actorOptions);
```

To create the DDPG agent, first specify the DDPG agent options using [rlDDPGAgentOptions](#).

```
agentOptions = rlDDPGAgentOptions;  
agentOptions.SampleTime = Ts;  
agentOptions.DiscountFactor = 0.99;  
agentOptions.MinibatchSize = 250;  
agentOptions.ExperienceBufferLength = 1e6;  
agentOptions.TargetSmoothFactor = 1e-3;  
agentOptions.NoiseOptions.MeanAttractionConstant = 1;  
agentOptions.NoiseOptions.Variance = 0.1;
```

Then, create the DDPG agent using the specified actor representation, critic representation, and agent options. For more information, see [rlDDPGAgent](#).

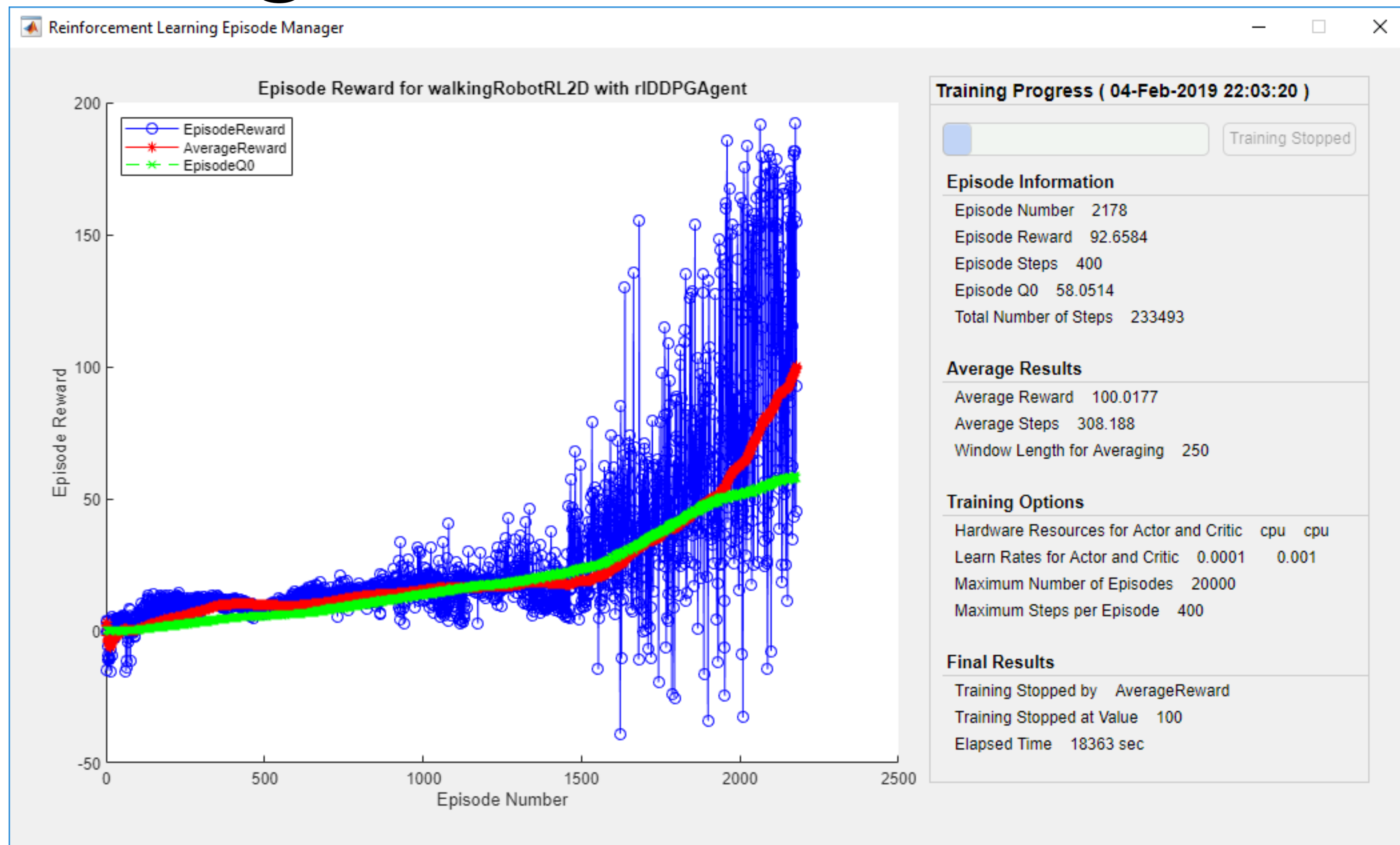
```
agent = rlDDPGAgent(actor,critic,agentOptions);
```

DDPGAgent comes with Reinforcement Learning Toolbox: <https://nl.mathworks.com/help/reinforcement-learning/ref/rlddpagent.html>



Training

4



Training - Options

```
maxEpisodes = 10000;
maxSteps = floor(Tf/Ts);
trainOpts = rlTrainingOptions(...
    'MaxEpisodes',maxEpisodes,...
    'MaxStepsPerEpisode',maxSteps,...
    'ScoreAveragingWindowLength',250,...
    'Verbose',false,...
    'Plots','training-progress',...
    'StopTrainingCriteria','AverageReward',...
    'StopTrainingValue',100,...
    'SaveAgentCriteria','EpisodeReward',...
    'SaveAgentValue',150);
```



Training - Options

```
trainOpts.UseParallel = false; %true  
trainOpts.ParallelizationOptions.Mode = 'async';  
trainOpts.ParallelizationOptions.StepsUntilDataIsSent = 32;  
trainOpts.ParallelizationOptions.DataToSendFromWorkers = 'Experiences';
```

MATLAB

Simulink

**Reinforcement
Learning Toolbox**

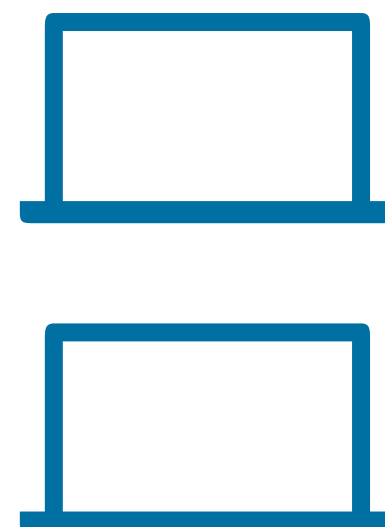
Simscape

**Deep Learning
Toolbox**

**Simscape
Multibody**

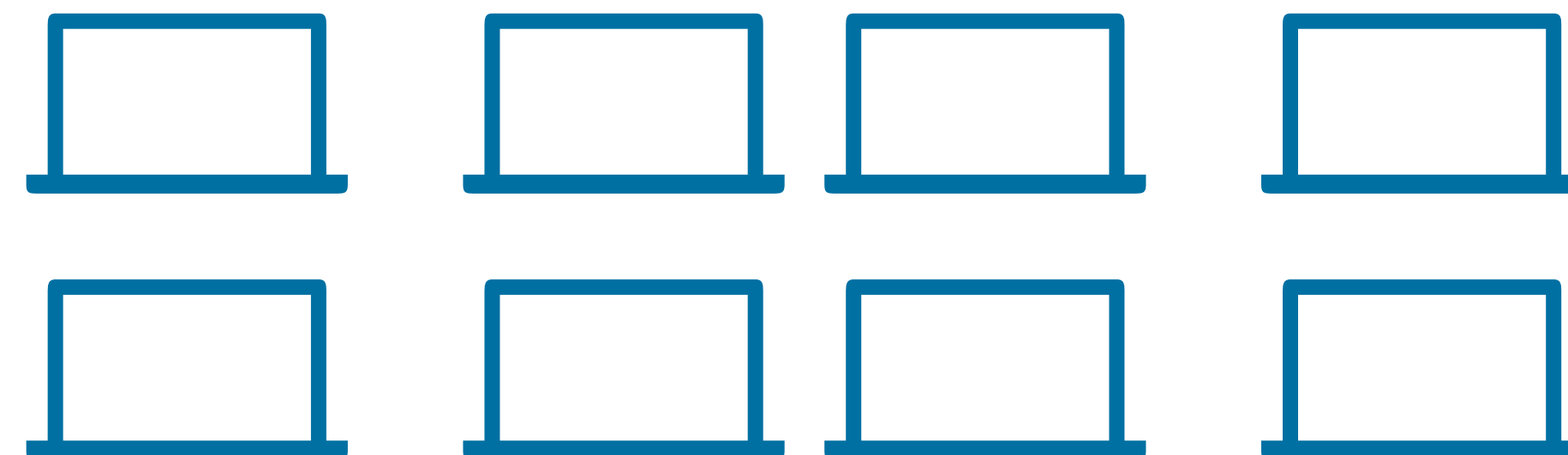
**Home Version
€ 119 MATLAB
€ 35 per toolbox**

**Run Simulations in // on
my machine leveraging
multiple cores**



MATLAB Parallel Computing Toolbox

**Run Simulations in // on
my cluster**



MATLAB Parallel Server



Putting it all together

4

Show 2 Minute Demo



Some drawbacks of this approach

- Unexplainable Network
- Pinpointing Problems - test -> redesign -> train -> test
- The Larger Problem
- Verifying the Learned Policy
- Formal Verification Methods



4 Things to Remember

- Reinforcement Learning is also possible with MATLAB
- Simulink & Simscape can help model the environment
- Not everything should be solved with Machine Learning
- Follow your passion and keep learning





VersionBay
embracing change